Ministry of High Education
Culture and Science city at Oct 6,
The High Institute of Computer Science & Information Systems

Graduation Project

"Smart Event: AI-Powered Multi-Sided Event Management, Recommendation & Logistics Marketplace"

Submitted by:

#	ID	Name
1	223373	كيرمينا ظريف خيرى جاد
2	223681	محمد محي السيد عبد العظيم
3	221267	يوسف عمرو سيد صالحين
4	220522	كريم عباس السيد عبد الغفار
5	220739	عبدالرحمن محمد سمير
6	220739	زياد أحمد شهاب الدين محمد
7	224051	فيصل فرج سعد عسيري

A graduation project submitted in partial fulfillment of the requirements for the degree of Bachelor of Computer Science and Information System.

Supervised by:

Dr.

Assistant

TA.

Project No: **Academic Year: 2025/ 2026**

Table of Contents

Acknowledgement

Abstract

Table of contents

List of figures

List of tables

List of abbreviations and acronyms

Chapter 1: introduction
1.1 Overview ……..5
1.2 Motivation
1.3 Objective
1.4 Aim
1.5 Scope …….....7
1.6 General constraints…… ….8
1.7 Organization of the project……...…….. 9
Chapter 2 Background and Previous work
2.1 Background……..……...……. 7
2.2 Previous work……...……...…… 7
Chapter 3 planning and analysis
3.1 Planning……..10
3.1.1 Feasibility study and estimated cost
3.1.2 Gantt chart
3.2 Analysis and limitations of existing system…….……..16
3.3 Need for new system…….……..……17
3.4 Analysis of new system…….……...……17
3.4.1 User Requirements …….…….16
3.4.2 System Requirements …….…….18
3.4.3 Domain Requirements …….……20
3.4.4 Functional Requirements ……..……21
3.4.5 Non- Functional Requirements …….…..……..27
3.5 Advantages of new system…….……..……26
3.6 User characteristics…….…….……27
Chapter 4: design
4.1 Design and Implementation Constraints…….……..28
4.2 Assumptions and dependencies…….……29
4.3 Risks and risk management…….……..30
4.4 Design of database ERD…..……...……38
4.1.1 Entity Relationship Diagram……38
4.1.2 Mapping of Entity Relationship Diagram……38
4.5 Class diagram……....41
4.6 Use case diagram…… 42
4.3.1 Primary Use-case Diagram……..42
4.3.2 Use-case Scenarios……...…42
4.7 Activity diagram……...…51
4.8 Sequence diagram…….……..……..59
4.9 State diagram……..59

Chapter 5: Implementation
5.1 Software Architecture……..……..……71
5.2 User Interface……..…….……71
5.3 Results and Discussion……72
Chapter 6: Testing
6.1 Unit Testing …….……73
6.2 Integration Testing……86
Conclusion…….……87
Future work……..….88
References…….……90
Appendices……**92**
ملخص المشروع بالعربى……93

CHAPTER 1

INTRODUCTION

1.1 Overview

The events industry involves a wide range of activities, including conferences, workshops, exhibitions, festivals, professional gatherings, entertainment events, and other types of physical, online, and hybrid events. Managing these events requires the coordination of multiple participants and processes, starting from event discovery and registration and extending to ticket management, attendee check-in, event organization, and the provision of operational services.

In many existing approaches, these activities are handled through separate platforms or communication channels. Attendees may use one platform to discover events and another process to complete registration or manage their tickets. Event organizers, on the other hand, are responsible for managing event information, schedules, speakers, sponsors, ticket categories, capacities, attendees, and entry operations. For events that require external services, organizers also need to communicate with execution companies that provide services such as sound systems, lighting, stages, security, catering, photography, and other event-related requirements.

This fragmentation creates difficulties for the different participants of the event ecosystem. Attendees may find it difficult to discover events that match their interests and requirements, while organizers may have difficulty coordinating both the public-facing and operational sides of their events. Execution companies may also lack a structured environment for receiving service requests and submitting quotations.

To address these challenges, **Smart Event: AI-Powered Multi-Sided Event Management, Recommendation & Logistics Marketplace** is proposed as an integrated event management ecosystem. The platform combines Business-to-Consumer (B2C) event discovery and ticketing with Business-to-Business (B2B) event organization and event execution services.

The proposed system connects four main actors: **Attendees, Event Organizers, Execution Companies, and Super Admins**. Attendees can discover events, receive personalized recommendations, book tickets, manage their bookings, and use digital QR tickets for event entry. Organizers can create and manage events, configure ticket types and capacities, manage schedules and speakers, monitor attendees, and request services from execution companies. Execution companies can create professional service profiles, complete the required verification process, receive Requests for Quotations (RFQs), and submit quotations

to organizers. The Super Admin is responsible for platform governance, verification, event approval, monitoring, and administrative operations.

Artificial Intelligence is an important component of the proposed ecosystem. A dedicated Python-based AI microservice is responsible for AI-related operations and communicates with the main backend through REST APIs. The AI layer is designed to support conversational event discovery, personalized event recommendations, event logistics planning, and intelligent matching between organizers and execution companies.

The initial implementation of Smart Event focuses on a web application. The architecture is designed to support a mobile application as a future extension while maintaining the same core backend services.

1.2 Motivation

The motivation for developing Smart Event comes from the increasing complexity of event organization and the fragmentation of the processes involved in managing events.

From the attendee perspective, traditional event discovery often requires users to search manually for events and compare different characteristics such as category, location, date, and price. This approach does not always provide personalized results or allow users to describe their requirements naturally.

From the organizer perspective, event management involves several independent responsibilities. An organizer may need to manage event information, ticket categories, capacities, schedules, speakers, sponsors, bookings, and attendee entry. In addition, organizers of larger events may need to identify and coordinate multiple execution companies, which can become difficult when requests, quotations, and communication are handled through unstructured channels.

Execution companies also require a structured marketplace where they can present their services, provide company information, receive relevant requests, submit quotations, and build a performance history through reviews.

These challenges motivate the development of a unified platform capable of connecting the different sides of the event ecosystem. The use of Artificial Intelligence provides an additional opportunity to improve the user experience and support decision-making.

The proposed system therefore aims to move beyond traditional event listing and ticketing systems by combining event management with intelligent discovery, recommendation, logistics assistance, and B2B service matching.

1.3 Objective

The main objectives of the Smart Event project are:

1. To develop a centralized platform for discovering and managing events.
2. To provide attendees with an efficient mechanism for searching and discovering events according to their interests and requirements.
3. To develop a conversational AI assistant capable of identifying important event-search parameters and asking for missing information when necessary.
4. To provide personalized event recommendations using relevant event and user information.
5. To allow organizers to create and manage physical, online, hybrid, and multi-day events.
6. To provide organizers with flexible ticket management, including multiple ticket types, organizer-defined capacities, cancellation policies, and waiting-list management.
7. To allow an attendee to book multiple tickets and quantities for multiple people within the same booking process.
8. To provide digital tickets containing unique QR-based identifiers for event entry.
9. To provide organizers with a QR scanning and check-in process for validating attendee tickets and preventing duplicate entry.
10. To provide structured speaker management, including speaker profiles, biographies, professional information, and supporting files such as CVs.
11. To provide a B2B marketplace connecting event organizers with execution companies.
12. To allow execution companies to create service profiles and respond to organizers' Requests for Quotations.
13. To provide AI-assisted logistics planning and intelligent vendor matching based on event requirements and available vendor information.
14. To provide a centralized notification mechanism for important system activities and events.
15. To provide administrative functionality for user management, company verification, event approval, monitoring, and platform governance.
16. To establish a modular architecture that supports future integrations such as mobile applications, external event APIs, and real payment gateways.

1.4 Aim

The primary aim of Smart Event is to develop an **AI-powered multi-sided event ecosystem** that integrates event discovery, event management, ticketing, attendee check-in, and event execution services into a single platform.

The system aims to reduce fragmentation between the different participants of the event industry and provide intelligent services that assist users in finding suitable events, managing events, planning operational requirements, and identifying appropriate execution companies.

The project also aims to establish a scalable technical foundation in which the core business logic is handled by a Node.js backend while AI functionality is isolated within an independent Python-based microservice.

1.5 Scope

The scope of the initial Smart Event implementation covers the development of a web-based platform and its supporting backend and AI services.

Attendee Scope

The attendee side of the system includes:

- User registration and authentication.
- Event browsing and searching.
- Event filtering and discovery.
- AI-assisted event search.
- Personalized event recommendations.
- Event details and schedules.
- Speaker information.
- Multiple ticket types.
- Booking multiple ticket types and quantities in one booking.
- Digital ticket management.
- QR-based ticket verification.
- Ticket cancellation according to organizer-defined policies.
- Waiting-list participation for fully booked events.
- Booking and attendance information.

- Notifications related to important user activities.

Event Organizer Scope

The organizer side includes:

- Organizer registration and account management.
- Event creation and management.
- Physical, online, and hybrid event support.
- Multi-day event support.
- Venue and schedule management.
- Ticket type creation.
- Ticket pricing and capacity configuration.
- Ticket cancellation policy configuration.
- Waiting-list management.
- Speaker management.
- Speaker biography and professional information.
- Speaker CV and supporting-file management.
- Sponsor management.
- Attendee management.
- QR-based attendee check-in.
- Event attendance monitoring.
- Creation of service requirements and RFQs.
- Receiving and comparing vendor quotations.
- Selection of suitable execution companies.

Execution Company Scope

The execution company side includes:

- Company registration.
- Company profile management.
- Service and service-category management.
- Uploading required company documentation.
- Verification workflow.

- Receiving relevant RFQs.
- Preparing and submitting quotations.
- Managing active service engagements.
- Receiving reviews and maintaining a performance history.

AI Scope

The AI layer includes:

- Conversational event discovery.
- Task-oriented information extraction.
- Identification of missing event-search parameters.
- Personalized recommendation support.
- Event logistics requirement estimation.
- Execution-company matching.
- Processing and structuring speaker information from supported CV files.

The AI functionality is implemented as an independent Python/FastAPI microservice that communicates with the main Node.js backend through REST APIs.

Administrative Scope

The Super Admin functionality includes:

- User management.
- Event approval.
- Execution-company verification.
- Monitoring platform activities.
- Managing administrative operations.
- Reviewing complaints and platform issues.
- Monitoring system and business information.
- Managing platform-level governance.

1.6 General Constraints

The development of Smart Event is subject to several technical, operational, and project constraints.

1.6.1 Payment Processing Constraint

Payment functionality is considered part of the system architecture; however, real payment gateway activation is not included in the current implementation. The payment workflow can therefore be represented through a processing or simulated mechanism until a real payment provider is integrated.

1.6.2 External Event Search Constraint

The external event discovery feature depends on the availability of a real and supported external event API. The functionality will remain inactive until an appropriate API is connected and configured.

The system must not represent external events as live data when no real external API is available.

1.6.3 Mobile Application Constraint

The initial implementation focuses on the web application. A mobile application is planned as a subsequent phase and is not required for the initial implementation.

1.6.4 LinkedIn Integration Constraint

Speaker information may be supported through professional profiles or CV files. Any future LinkedIn integration must depend on an official and technically available API rather than unauthorized data extraction.

1.6.5 Data Storage Constraint

Uploaded images and files are stored in the application's upload storage, while the database stores the corresponding file paths or references required by the application.

1.6.6 AI Data Constraint

The quality of AI recommendations, matching, and information extraction depends on the quality and availability of the data provided to the AI services.

1.6.7 Development Time Constraint

As a graduation project, development must remain within the available academic timeframe. Therefore, advanced features that are not essential to the core objectives may be deferred to future work.

1.6.8 Technology Constraint

The initial web system uses a predefined technology stack consisting of React.js and TypeScript for the frontend, Node.js, Express.js, and TypeScript for the main backend, MySQL with Prisma ORM for data management, and Python with FastAPI for the AI microservice.

1.6.9 Integration Constraint

The core backend is responsible for the main business logic, while AI functionality is separated into an independent service. Communication between these components is performed through REST APIs.

1.7 Organization of the Project

This project report is organized into six main chapters.

Chapter One — Introduction presents the Smart Event project, its motivation, objectives, overall aim, scope, and general constraints.

Chapter Two — Background and Previous Work presents the theoretical background required to understand the proposed system and reviews previous and existing systems related to event management, event discovery, ticketing, recommendation, and event-service marketplaces.

Chapter Three — Planning and Analysis presents the project planning process, feasibility study, estimated cost, development schedule, analysis of existing systems, identified limitations, need for the proposed system, user requirements, system requirements, domain requirements, functional requirements, and non-functional requirements.

Chapter Four — Design presents the design of the proposed system, including implementation constraints, assumptions and dependencies, risk management, database design, Entity Relationship Diagram, ERD mapping, Class Diagram, Use Case Diagram and scenarios, Activity Diagrams, Sequence Diagrams, and State Diagrams.

Chapter Five — Implementation describes the software architecture, implementation technologies, user interface, major system components, and the resulting implementation of the proposed platform.

Chapter Six — Testing presents the testing activities performed on the system, including unit testing and integration testing, and evaluates whether the implemented system satisfies the defined requirements.

The report is followed by the **Conclusion**, **Future Work**, **References**, and **Appendices**, which provide the final conclusions, possible future extensions, cited sources, and supporting project materials.

CHAPTER 2

BACKGROUND AND PREVIOUS WORK

2.1 Background

The development of modern event management platforms requires the integration of several technical and business concepts. Event management is no longer limited to publishing event information and selling tickets. A complete event ecosystem may involve event discovery, registration, ticket management, attendee access control, event scheduling, speaker management, service procurement, communication, and administrative supervision.

Smart Event combines these concepts within a single multi-sided platform. The following background areas provide the foundation for understanding the proposed system.

2.1.1 Event Management Systems

An Event Management System (EMS) is a software platform designed to support the planning, organization, management, and monitoring of events. Depending on its capabilities, an EMS may provide features such as event creation, registration, scheduling, ticketing, attendee management, communication, reporting, and event access control.

Traditional event platforms generally focus on the public-facing side of an event, particularly event publishing, registration, and ticket sales. However, the operational side of event organization may involve additional participants such as service providers, equipment suppliers, security companies, catering companies, and other execution businesses.

Smart Event extends the traditional event management concept by combining attendee-facing services with organizer and execution-company workflows.

2.1.2 Event Discovery and Recommendation

Event discovery refers to the process through which users search for and identify events that may be relevant to their interests.

A basic event discovery system may allow users to search using predefined filters such as:

Event category.

Location.

Date.

Price.

Event format.

However, users may not always know the exact filters they need. They may instead describe their requirements in natural language.

For this reason, Smart Event introduces an AI-assisted conversational discovery mechanism. The system is designed to identify relevant information from the user's request and determine which required parameters are available and which parameters are missing.

The main parameters considered by the proposed system include:

Event domain or category.

Event format.

Location.

Budget.

Date or time range.

When required information is missing, the conversational component can request the missing information before generating a structured event search.

2.1.3 Recommendation Systems

Recommendation systems are software systems that help users discover items that are potentially relevant to them based on available information.

In an event platform, recommendation can be based on multiple factors, including:

Event category.

Location.

Price.

Date.

User preferences.

Previous interaction or booking history.

Event popularity.

Event rating.

Smart Event uses a hybrid recommendation concept rather than depending on a single factor. A recommendation score can combine multiple weighted criteria to determine the relevance of an event to a particular user.

A conceptual representation of the scoring process is:

where:

represents category relevance.

represents location relevance.

represents price compatibility.

represents date compatibility.

represents user history.

represents popularity.

represents rating.

represent the weights assigned to the individual factors.

The actual weights and evaluation methodology will be defined during the analysis and implementation stages.

2.1.4 Artificial Intelligence in Event Management

Artificial Intelligence can support event platforms by improving search, recommendation, classification, prediction, and decision-support processes.

In Smart Event, AI is not responsible for the core business operations of the platform. Instead, AI functionality is separated into an independent microservice.

The AI layer is implemented using Python and FastAPI, while the main business backend is implemented using Node.js and Express.js. The two components communicate through REST APIs.

This separation allows AI-related functionality to evolve independently from the main business logic.

The initial AI scope includes:

Conversational event discovery.

Slot or parameter extraction.

Personalized event recommendation.

Event logistics requirement estimation.

Execution-company matching.

Supporting speaker-information extraction from available CV files.

2.1.5 Task-Oriented Dialogue and Slot Filling

A task-oriented dialogue system is designed to accomplish a specific task through interaction with a user.

In Smart Event, the task is to identify events that satisfy the user's requirements.

The conversational process can be represented as:

User Request → Information Extraction → Missing Information Detection → Follow-up Questions → Structured Search → Recommended Events

For example, a user may specify an event category, city, and budget but not provide a date. The system can identify the missing date and request it before proceeding.

This approach provides a more natural interaction mechanism than requiring users to manually configure every search filter.

2.1.6 B2B Service Marketplace

A B2B marketplace connects businesses that require services with businesses that provide those services.

Within Smart Event, the B2B marketplace connects event organizers with execution companies.

Execution companies can provide services such as:

Sound systems.

Lighting.

Stages.

Screens.

Security.

Catering.

Photography.

Other event-related services.

The platform introduces a structured Request for Quotation (RFQ) workflow. Instead of relying only on informal communication, an organizer can define service requirements and submit them to suitable execution companies.

Companies can then review the request and submit quotations.

The general workflow is:

Organizer Requirements → RFQ → Vendor Matching → Quotations → Organizer Comparison → Vendor Selection

---

2.1.7 AI-Based Vendor Matching

Matching organizers with suitable execution companies is a multi-criteria decision problem.

The proposed system considers information such as:

Required service.

Company capabilities.

Geographic availability.

Previous performance.

Rating.

Estimated cost.

Availability or readiness.

The AI layer can use these criteria to rank suitable execution companies according to their compatibility with the organizer's requirements.

The resulting match percentage should be treated as a system-generated decision-support indicator rather than a guaranteed measure of service quality.

2.1.8 Digital Ticketing and QR-Based Check-in

Digital ticketing enables attendees to receive electronic tickets instead of relying on physical tickets.

Smart Event uses unique ticket identifiers associated with individual bookings and attendees. A QR code is generated for the digital ticket and can be scanned at the event entrance.

The verification process checks relevant ticket information before allowing entry.

The proposed validation process includes checking whether:

The ticket exists.

The booking is confirmed.

The ticket belongs to the selected event.

The ticket has not already been used.

The ticket is valid according to the event's access rules.

After successful verification, the check-in status is updated and the attendance record is stored.

This mechanism provides the organizer with a structured method for monitoring event attendance and reducing duplicate entry.

2.1.9 Speaker Management

Speakers are an important part of many conferences, workshops, and professional events.

Smart Event therefore treats speakers as persistent entities within the system rather than storing them only as temporary event information.

A speaker profile can contain information such as:

Name.

Professional biography.

Profile image.

Professional information.

Experience.

Related event participation.

Supporting documents such as a CV.

The system can maintain speaker information in the database and associate speakers with one or more events.

Where technically available and legally appropriate, speaker information may also be supported through external professional-profile integrations. However, such integrations depend on official APIs and are not assumed to be active without an actual supported integration.

2.1.10 Multi-Day Events and Ticket Management

Events may take place over multiple days and may contain multiple schedules or sessions.

Smart Event therefore supports multi-day events and allows organizers to configure the event schedule accordingly.

The ticketing model also supports multiple ticket categories. An attendee can select more than one ticket type and different quantities within a single booking process.

The platform also supports:

Organizer-defined ticket capacity.

Ticket cancellation according to event policy.

Waiting lists for fully booked events.

Multiple attendees within a booking.

These capabilities allow the ticketing process to represent more realistic event-registration scenarios.

2.1.11 Notification Systems

A multi-sided platform requires users to be informed about important changes and actions.

Smart Event includes a notification mechanism for relevant system events, such as:

Booking confirmation.

Booking cancellation.

Ticket updates.

Waiting-list status.

Event updates.

RFQ updates.

Quotation updates.

Vendor-related actions.

Administrative actions.

The implementation uses an external notification library/service rather than building a complete notification delivery infrastructure from scratch.

The exact notification channels depend on the selected service and its supported integrations.

2.2 Previous Work

Several existing platforms provide important parts of the functionality targeted by Smart Event. However, they generally focus on a particular side of the event or service-management process rather than combining all required components into one ecosystem.

The previous-work analysis therefore focuses on identifying the capabilities of representative platforms and determining the gap that motivates the proposed system.

2.2.1 Eventbrite

Eventbrite is an event platform focused primarily on event discovery, event publishing, registration, and ticketing.

Its model demonstrates the importance of providing users with an accessible event-discovery and ticket-purchasing experience.

However, the main concept targeted by Smart Event goes beyond attendee-facing event discovery and ticketing by incorporating a B2B marketplace for event execution services and AI-assisted logistics and vendor matching.

2.2.2 Meetup

Meetup focuses on helping people discover and participate in groups and events based on shared interests.

Its model demonstrates the value of interest-based event discovery and community participation.

However, Smart Event addresses a broader event-management workflow that includes ticket management, organizer operations, execution-company services, RFQs, quotations, and administrative governance.

2.2.3 Tazkarti

Tazkarti represents a ticketing-oriented approach in which users can access information about events and purchase tickets.

This type of system demonstrates the importance of digital ticketing and organized event access.

Smart Event expands the concept by integrating ticketing with event creation, organizer management, AI recommendations, QR-based check-in, and B2B event execution services.

2.2.4 Thumbtack

Thumbtack represents a service marketplace model that connects customers with professionals who provide different types of services.

This model is relevant to Smart Event because the proposed B2B marketplace similarly connects a party requiring services with companies capable of providing those services.

However, Smart Event specializes this marketplace model for the event industry by connecting event organizers with execution companies and structuring the process around event-specific requirements, RFQs, quotations, and vendor matching.

2.2.5 Comparison of Previous Work

Capability	Eventbrite	Meetup	Tazkarti	Thumbtack	Smart Event
Event Discovery	✓	✓	✓	—	✓

Event Ticketing	✓	**Limited/Varies**	✓	—	✓
Event Organizer Management	✓	✓	**Limited**	—	✓
Digital Ticketing	✓	**Limited/Varies**	✓	—	✓
QR Check-in	**Supported/Depends on configuration**	—	—/**Depends on service**	—	✓
AI Conversational Discovery	**Not core**	**Not core**	**Not core**	**Not core**	✓
Personalized AI Recommendation	**Limited/Platform-dependent**	**Interest-based**	**Limited**	**Matching-focused**	✓
Multi-Day Events	✓	✓	**Event-dependent**	—	✓
Multiple Ticket Types	✓	**Event-dependent**	✓	—	✓
Waiting List	**Platform-dependent**	**Platform-dependent**	**Event-dependent**	—	✓
B2B Event Execution Marketplace	—	—	—	**✓ General Services**	**✓ Event-Specific**
RFQ and Quotations	—	—	—	**Service-oriented**	✓
AI Vendor Matching	—	—	—	**Matching-oriented**	✓
Speaker Profiles	**Event-dependent**	**Limited**	**Event-dependent**	—	✓
Event Logistics Planning	—	—	—	**General Services**	✓
Super Admin Governance	✓	✓	✓	✓	✓

The following conceptual comparison summarizes the major differences between the representative systems and Smart Event.

Note: The comparison is a conceptual analysis of platform capabilities relevant to this project. Exact feature availability can vary by product version, region, configuration, or service plan and should be verified against official documentation before final submission.

2.2.6 Identified Gap

The analysis of previous work indicates that existing platforms provide valuable solutions for specific parts of the event ecosystem.

Event platforms primarily address event discovery, registration, and ticketing. Community-oriented platforms emphasize event discovery and participation. Service marketplaces focus on connecting customers with service providers.

The identified gap addressed by Smart Event is the integration of these different functions within a single event-focused ecosystem, while adding AI-based decision support.

The proposed system combines:

B2C

Event discovery.

AI-assisted search.

Personalized recommendations.

Ticket booking.

Digital tickets.

QR check-in.

with:

B2B

Event organization.

Event logistics planning.

Execution-company profiles.

RFQs.

Quotations.

Vendor matching.

Reviews and performance information.

The system therefore aims to provide a unified workflow:

Discover → Book → Attend

for attendees, and:

Create → Plan → Request Services → Compare Quotations → Execute → Monitor

for organizers.

The separation of the AI layer into an independent microservice also provides an architectural foundation for developing intelligent capabilities without tightly coupling them to the core business backend.

2.2.7 Summary

The background presented in this chapter establishes the technical and conceptual foundations of Smart Event, including event management, event discovery, recommendation systems, conversational AI, B2B marketplaces, vendor matching, digital ticketing, QR-based check-in, speaker management, and notification systems.

The review of previous work shows that existing platforms provide strong solutions for individual parts of this ecosystem. However, the proposed Smart Event platform focuses on integrating attendee services, organizer operations, and event execution services into one platform, while introducing AI-supported discovery, recommendation, logistics planning, and vendor matching

CHAPTER 3

PLANNING AND ANALYSIS

3.1 Planning

3.1.1 Feasibility Study and Estimated Cost

The feasibility study evaluates whether the proposed Smart Event system can be developed and operated using the available technical resources, development time, and project requirements.

The study considers four main aspects: technical feasibility, operational feasibility, economic feasibility, and schedule feasibility.

Technical Feasibility

The proposed system is technically feasible because the required technologies are widely supported and can be integrated into a unified web-based architecture.

The frontend is based on React.js, TypeScript, Tailwind CSS, HTML5, CSS3, and Vite. The main backend uses Node.js, Express.js, TypeScript, RESTful APIs, and Socket.IO. MySQL is used as the relational database with Prisma ORM.

AI-related functionality is separated from the core backend and implemented as an independent Python microservice using FastAPI. Communication between the main backend and the AI service is performed through REST APIs.

The proposed architecture therefore allows the team to develop the core business logic independently from the AI components.

Operational Feasibility

The system is operationally feasible because it provides different interfaces and workflows for the four main actors:

Attendee.

Event Organizer.

Execution Company.

Super Admin.

Each actor receives functionality related to their responsibilities within the event ecosystem.

The platform also provides a centralized notification mechanism for important system activities.

Economic Feasibility

The project is primarily developed using open-source technologies and development tools. Therefore, the initial software-development cost is expected to remain limited.

The main potential costs are associated with infrastructure and optional third-party services, such as:

Hosting.

Database hosting.

File storage.

Notification services.

External event APIs.

Future payment gateway services.

Domain and deployment services.

Real payment processing and external event discovery APIs are not activated in the initial implementation unless the required external services are available.

Schedule Feasibility

The system is divided into independent modules to make development manageable within the graduation-project timeframe.

The major development activities include:

Requirements analysis.

System design.

Database design.

Backend development.

Frontend development.

AI microservice development.

Integration.

Testing.

Documentation.

Final presentation and demonstration.

3.1.2 Gantt Chart

The development process follows an iterative approach in which requirements, design, implementation, integration, and testing are performed progressively.

The project schedule should cover the following major activities:

Activity	Main Deliverable
Project Definition	Final project scope
Requirements Analysis	Requirements specification
Background Research	Previous-work analysis
System Design	Architecture and UML
Database Design	ERD and database schema
Backend Development	REST APIs and business logic
Frontend Development	Web application
AI Development	AI microservice
System Integration	Integrated platform
Testing	Test cases and results
Documentation	Final project report
Presentation Preparation	Presentation and demo

Note: The exact dates and durations should be taken from the team's actual academic schedule and should not be invented in the final report.

3.2 Analysis and Limitations of Existing System

Existing event-management solutions generally provide functionality for event publishing, discovery, registration, or ticketing. Some platforms focus on community-based event discovery, while service marketplaces focus on connecting customers with service providers.

Although these systems provide useful functionality, the processes involved in a complete event lifecycle may remain distributed across different platforms and communication channels.

Several limitations can therefore be identified from the perspective of the proposed Smart Event ecosystem.

3.2.1 Fragmented Event Workflow

Event discovery, ticketing, event management, and operational service procurement may be handled through different systems.

This fragmentation can increase the effort required from organizers and attendees.

3.2.2 Limited Intelligent Discovery

Traditional event-search mechanisms often depend primarily on manually selected filters.

This limits the ability of users to express their requirements naturally and receive recommendations based on multiple factors.

3.2.3 Separation Between Event Management and Service Procurement

Event organizers may require several execution services but these services are not necessarily integrated into the same event-management workflow.

As a result, organizers may depend on external communication channels to request quotations and compare service providers.

3.2.4 Limited AI-Based Vendor Matching

General service marketplaces can connect customers with providers, but an event-specific platform can use event requirements to improve the matching process.

Smart Event addresses this through an event-oriented vendor matching mechanism.

3.2.5 Limited Integration of Event Execution

Managing tickets and discovering events does not necessarily cover operational requirements such as:

Sound.

Lighting.

Stages.

Security.

Catering.

Photography.

Other execution services.

The proposed system treats these services as part of the event lifecycle.

3.3 Need for New System

The identified limitations demonstrate the need for an integrated platform that connects the different sides of the event ecosystem.

Smart Event is proposed to provide a centralized environment where attendees can discover and book events while organizers can manage their events and obtain the services required for execution.

The proposed system addresses this need through four connected actor groups:

Attendees

They require a simple mechanism for:

Discovering relevant events.

Searching using natural language.

Receiving personalized recommendations.

Booking tickets.

Managing digital tickets.

Checking event information.

Accessing events through QR-based check-in.

Event Organizers

They require tools for:

Creating events.

Managing event schedules.

Managing speakers.

Managing sponsors.

Creating ticket categories.

Defining ticket capacities.

Managing bookings.

Managing attendees.

Requesting execution services.

Comparing quotations.

Monitoring event entry.

Execution Companies

They require a structured B2B environment for:

Presenting their services.

Managing company profiles.

Completing verification.

Receiving RFQs.

Submitting quotations.

Managing active engagements.

Receiving reviews.

Super Admin

The platform requires administrative governance for:

User management.

Event approval.

Execution-company verification.

Platform monitoring.

Dispute management.

Administrative control.

3.4 Analysis of New System

The proposed system is analyzed according to the requirements of its users, technical environment, domain, and functional behavior.

3.4.1 User Requirements

UR-01 — Attendee Registration

The system shall allow an attendee to create and manage an account.

UR-02 — Event Discovery

The attendee shall be able to browse and search available events.

UR-03 — Intelligent Event Search

The attendee shall be able to interact with the AI assistant to describe event requirements.

UR-04 — Personalized Recommendations

The system shall provide event recommendations based on available user and event information.

UR-05 — Event Booking

The attendee shall be able to book tickets for an event.

UR-06 — Multiple Ticket Types

The attendee shall be able to select multiple ticket types within the same booking.

UR-07 — Multiple Attendees

The attendee shall be able to book tickets for multiple people according to the event's available capacity and rules.

UR-08 — Ticket Cancellation

The attendee shall be able to cancel eligible tickets according to the organizer-defined cancellation policy.

UR-09 — Waiting List

The attendee shall be able to join the waiting list when an event or ticket type reaches its configured capacity.

UR-10 — Digital Ticket

The system shall provide a digital ticket containing a unique QR identifier.

UR-11 — Event Check-in

The organizer shall be able to scan and validate attendee QR tickets.

UR-12 — Event Creation

The organizer shall be able to create and manage events.

UR-13 — Multi-Day Events

The organizer shall be able to create events extending over multiple days.

UR-14 — Speaker Management

The organizer shall be able to add speakers and maintain their information.

UR-15 — Speaker Documents

The system shall support storing speaker-related files such as CVs.

UR-16 — Service Requests

The organizer shall be able to create service requirements and RFQs.

UR-17 — Vendor Quotations

Execution companies shall be able to submit quotations for suitable RFQs.

UR-18 — Vendor Verification

The Super Admin shall be able to review and approve execution-company verification requests.

UR-19 — Notifications

The system shall notify users about relevant system activities.

UR-20 — Administrative Governance

The Super Admin shall be able to manage and monitor the platform.

3.4.2 System Requirements

The system is divided into the following major components:

Frontend

React.js.

TypeScript.

Tailwind CSS.

HTML5.

CSS3.

Vite.

Main Backend

Node.js.

Express.js.

TypeScript.

RESTful API.

Socket.IO.

The backend is responsible for the main business logic, including:

Users.

Authentication.

Events.

Bookings.

Tickets.

QR codes.

Execution companies.

Service requests.

Quotations.

Notifications.

Payments processing workflow.

Administration.

Database

MySQL.

Prisma ORM.

AI Microservice

Python 3.x.

FastAPI.

AI/ML libraries according to the implemented AI requirements.

The AI service communicates with the Node.js backend through REST APIs.

File Storage

Uploaded files and images are stored in the application's upload storage, while the database maintains the corresponding file paths or references.

3.4.3 Domain Requirements

The system operates within the event-management and event-services domain.

The major domain entities include:

Users

Users represent the accounts participating in the system.

Attendees

Attendees use the platform to discover and book events.

Organizers

Organizers create and manage events.

Execution Companies

Execution companies provide event-related services.

Events

Events contain information such as:

Name.

Description.

Category.

Format.

Venue.

Schedule.

Capacity.

Speakers.

Sponsors.

Ticket types.

Tickets

Tickets represent the access rights purchased by attendees.

Bookings

Bookings represent ticket reservations and may contain multiple ticket types and quantities.

Check-ins

Check-ins represent the successful validation of a ticket at the event entrance.

Services

Services represent event-execution capabilities provided by execution companies.

Service Requests / RFQs

RFQs represent organizer requests for specific event services.

Quotations

Quotations represent vendor responses to RFQs.

Reviews

Reviews represent feedback about completed event-service engagements.

3.4.4 Functional Requirements

The following functional requirements define the main behaviors expected from the system.

ID	Functional Requirement
FR-01	The system shall support user registration and authentication.
FR-02	The system shall support role-based access control.
FR-03	The system shall allow attendees to browse available events.
FR-04	The system shall allow attendees to search and filter events.
FR-05	The system shall provide AI-assisted event discovery.

ID	Functional Requirement
FR-06	The system shall identify missing search parameters during AI conversations.
FR-07	The system shall generate event recommendations.
FR-08	The system shall allow organizers to create and update events.
FR-09	The system shall support multi-day events.
FR-10	The system shall allow organizers to create multiple ticket types.
FR-11	The system shall allow organizers to define ticket capacity.
FR-12	The system shall support cancellation according to event policy.
FR-13	The system shall support waiting lists.
FR-14	The system shall allow attendees to purchase multiple ticket types in one booking.
FR-15	The system shall support multiple ticket quantities for multiple attendees.
FR-16	The system shall generate unique digital tickets.
FR-17	The system shall generate QR identifiers for tickets.
FR-18	The system shall validate QR tickets during check-in.
FR-19	The system shall prevent reuse of a successfully checked-in ticket.

ID	Functional Requirement
FR-20	The system shall record check-in information.
FR-21	The system shall allow organizers to manage speakers.
FR-22	The system shall store speaker-related files and references.
FR-23	The system shall allow organizers to create service requests.
FR-24	The system shall allow execution companies to receive relevant RFQs.
FR-25	The system shall allow execution companies to submit quotations.
FR-26	The system shall support vendor matching.
FR-27	The system shall support execution-company verification.
FR-28	The system shall support reviews and vendor performance information.
FR-29	The system shall provide system notifications.
FR-30	The system shall provide administrative management capabilities.

3.4.5 Non-Functional Requirements

NFR-01 — Security

The system shall protect user accounts and system resources through authentication, authorization, secure password storage, and role-based access control.

NFR-02 — Performance

The system should provide acceptable response times for normal user operations and should avoid unnecessary delays in common workflows.

NFR-03 — Availability

The deployed system should remain available to authorized users during its intended operating period.

NFR-04 — Scalability

The architecture should allow additional users, events, services, and AI capabilities to be added without requiring a complete redesign.

NFR-05 — Maintainability

The system should use modular components to simplify maintenance and future development.

NFR-06 — Reliability

The system should maintain consistent booking, ticket, and check-in information and should prevent invalid state transitions.

NFR-07 — Usability

The user interface should provide clear workflows appropriate to each system actor.

NFR-08 — Data Integrity

The database shall maintain consistent relationships between users, events, bookings, tickets, payments, RFQs, quotations, and related entities.

NFR-09 — Extensibility

The architecture should support future integrations such as:

Mobile applications.

External event APIs.

Real payment gateways.

Additional AI models.

Additional notification channels.

NFR-10 — Interoperability

The main backend and AI microservice shall communicate using defined REST API interfaces.

3.5 Advantages of New System

The proposed Smart Event system provides several advantages.

3.5.1 Unified Event Ecosystem

The platform combines attendee-facing and business-facing workflows within one ecosystem.

3.5.2 Intelligent Event Discovery

The conversational AI mechanism allows users to express their requirements more naturally.

3.5.3 Personalized Recommendations

Multiple event and user factors can be combined to generate more relevant recommendations.

3.5.4 Integrated B2B Marketplace

Organizers can request event services and receive quotations without leaving the platform.

3.5.5 Intelligent Vendor Matching

Execution companies can be ranked according to their compatibility with event requirements.

3.5.6 Structured Event Execution

The system connects event creation with operational service requirements.

3.5.7 Flexible Ticket Management

The system supports multiple ticket categories, capacities, cancellations, waiting lists, and multi-person bookings.

3.5.8 Controlled Event Access

QR-based ticket validation provides a structured mechanism for event check-in and attendance tracking.

3.5.9 Administrative Governance

Super Admin functionality provides centralized control over verification, approvals, and platform operations.

3.5.10 Extensible Architecture

The separation between the main backend and AI microservice provides a foundation for future expansion.

3.6 User Characteristics

Smart Event contains four primary user categories.

3.6.1 Attendee

The attendee is a general platform user who searches for and participates in events.

The attendee should be able to:

Navigate the web interface.

Search for events.

Use basic filters.

Interact with the AI assistant.

Complete bookings.

Access digital tickets.

Present QR tickets during check-in.

No advanced technical knowledge should be required.

3.6.2 Event Organizer

The Event Organizer is responsible for creating and managing events.

The organizer should have sufficient knowledge of:

Event configuration.

Ticket management.

Schedule management.

Attendee management.

Service requirements.

The organizer also uses the B2B marketplace to communicate service requirements and evaluate vendor quotations.

3.6.3 Execution Company

The Execution Company represents a business providing event-related services.

The company user should be able to:

Manage the company profile.

Add services.

Review RFQs.

Submit quotations.

Manage active engagements.

Review performance information.

The company may also be required to provide business documentation during the verification process.

3.6.4 Super Admin

The Super Admin has the highest administrative privileges within the platform.

The administrator is responsible for:

User governance.

Event approval.

Company verification.

Monitoring.

Administrative actions.

Dispute handling.

Because the Super Admin has privileged access, administrative operations must be protected through strong authentication and role-based authorization.

CHAPTER 3 — END

PROJECT STATUS

Chapter: 3 — Planning and Analysis
Status: Draft — Needs Revision

Completed

Planning

Feasibility Study

Cost structure

Gantt Chart structure

Existing-system limitations

Need for the new system

User requirements

System requirements

Domain requirements

Functional requirements

Non-functional requirements

Advantages

User characteristics

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction

This chapter presents the detailed design of the proposed Smart Event system. The design translates the requirements identified in Chapter 3 into a structured technical architecture that defines the major system components, their responsibilities, interactions, data structures, and user interfaces.

The proposed system follows a modular architecture consisting of a web frontend, a core backend, a relational database, and an independent AI microservice.

The design also considers the four main system actors: Attendee, Event Organizer, Execution Company, and Super Admin.

4.2 System Architecture

Smart Event follows a layered architecture in which each major component has a defined responsibility.

The main components are:

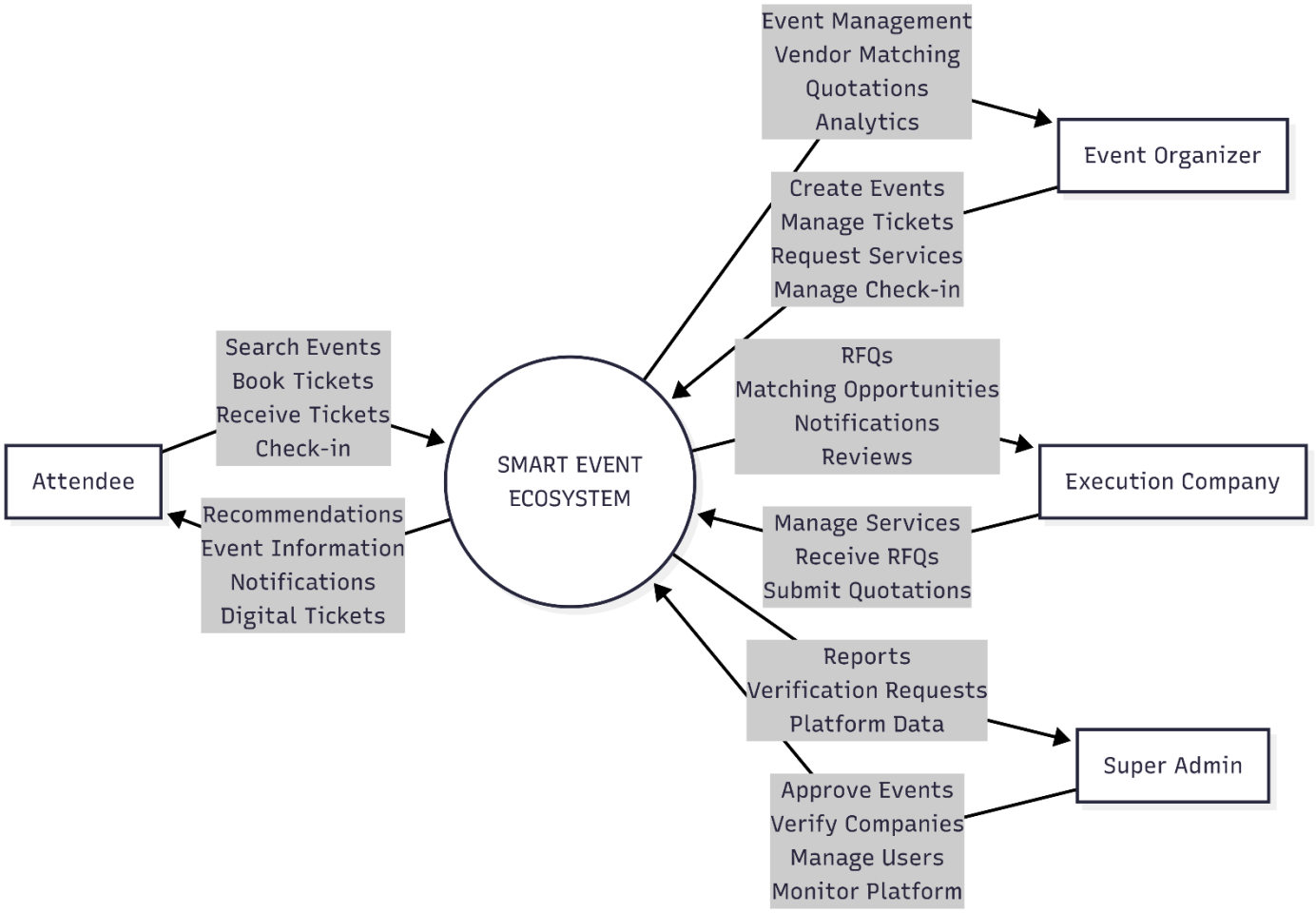
1. **Frontend Application.**
2. **Main Backend API.**
3. **Database Layer.**
4. **AI Microservice.**
5. **External Services and APIs.**
6. **File Storage.**

The frontend communicates with the main backend through secure REST APIs and Socket.IO where real-time communication is required.

The backend manages the core business logic and communicates with the MySQL database through Prisma ORM.

AI-related functionality is isolated in a Python FastAPI microservice and accessed by the Node.js backend through REST APIs.

Figure 4.1 — Overall System Architecture



4.3 Architectural Components

4.3.1 Frontend Layer

The frontend is responsible for presenting the system interface to users.

The web application is implemented using:

- **React.js.**
- **TypeScript.**
- **Tailwind CSS.**
- **HTML5.**
- **CSS3.**
- **Vite.**

The frontend provides different interfaces according to the user's role.

The main interfaces include:

- **Attendee interface.**

- **Organizer dashboard.**
- **Execution Company dashboard.**
- **Super Admin dashboard.**

4.3.2 Core Backend Layer

The backend represents the central business layer of the system.

It is implemented using:

- **Node.js.**
- **Express.js.**
- **TypeScript.**
- **RESTful APIs.**
- **Socket.IO.**

The backend is responsible for:

- **Authentication.**
- **Authorization.**
- **User management.**
- **Event management.**
- **Booking management.**
- **Ticket management.**
- **QR verification.**
- **Organizer operations.**
- **Execution-company operations.**
- **RFQs.**
- **Quotations.**
- **Notifications.**
- **Payment-processing workflow.**
- **Administrative operations.**

4.3.3 AI Microservice

AI functionality is separated from the core backend into an independent microservice.

The AI service is implemented using:

- **Python.**
- **FastAPI.**

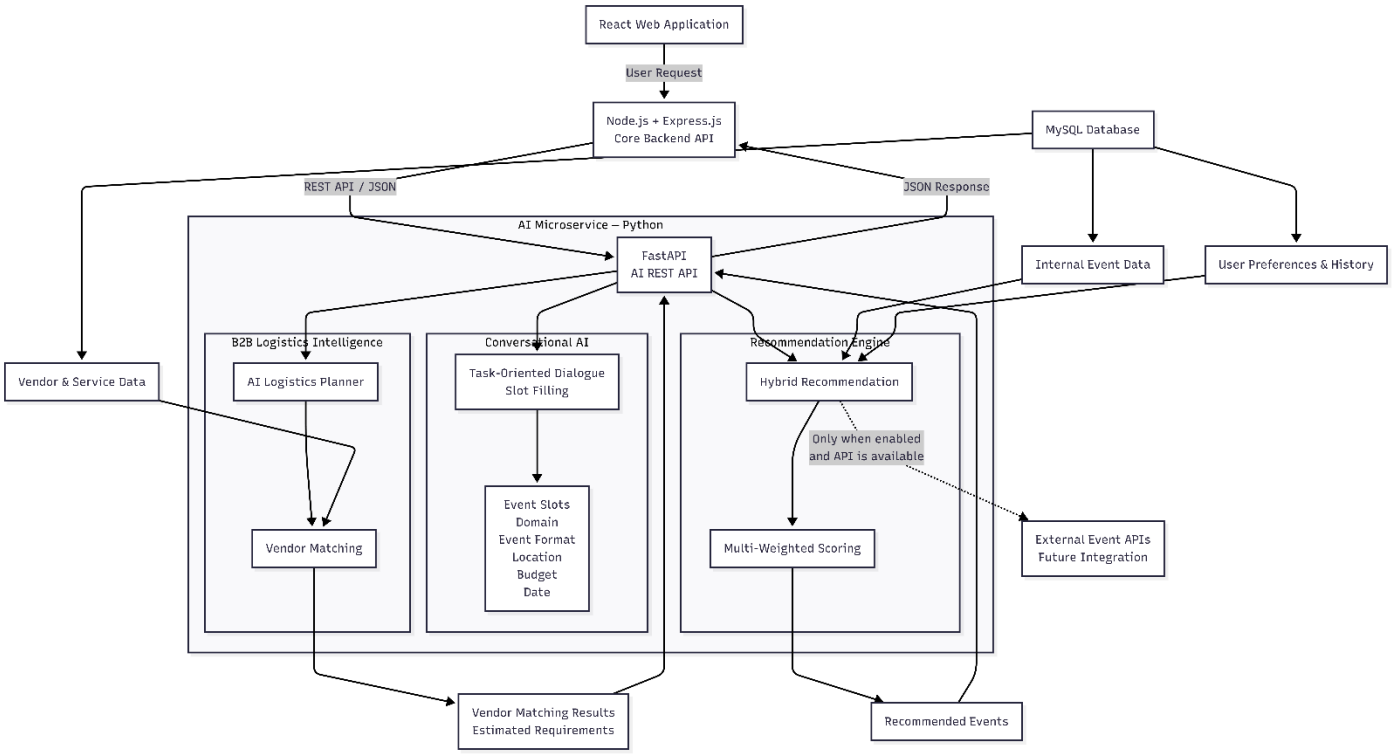
- **Appropriate AI/ML libraries.**

The microservice provides functionality such as:

- **Conversational event discovery.**
- **Slot extraction.**
- **Event recommendation.**
- **Logistics requirement estimation.**
- **Vendor matching.**
- **Supporting speaker-data extraction.**

The Node.js backend communicates with the AI microservice using REST APIs.

Figure 4.2 — AI Microservice Architecture



4.4 Communication Architecture

The system uses different communication mechanisms depending on the type of operation.

REST API

REST APIs are used for standard request-response operations such as:

- **Login.**
- **Event retrieval.**
- **Event creation.**
- **Booking.**
- **Ticket management.**

- **RFQ creation.**
- **Quotation submission.**
- **Administrative operations.**

Socket.IO

Socket.IO is used for real-time updates where required.

Examples include:

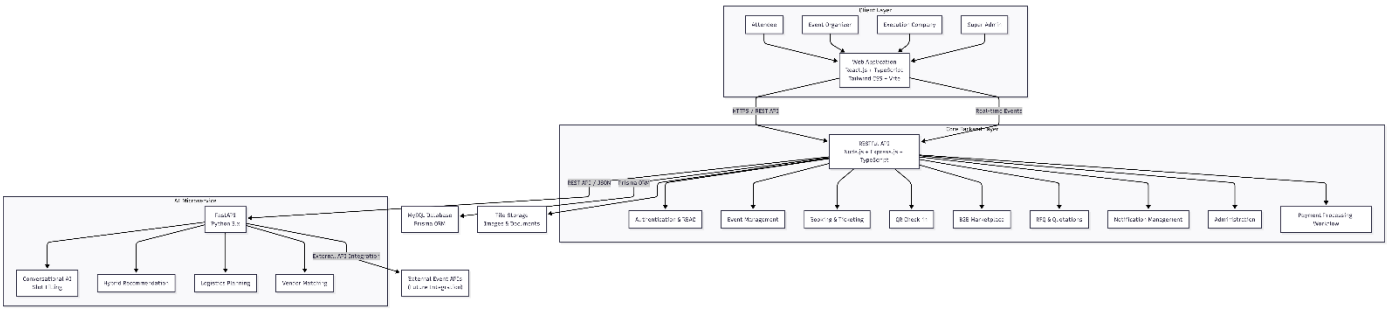
- **Live check-in counters.**
- **Real-time notifications.**
- **Booking-related updates.**
- **RFQ and quotation updates.**

AI REST Communication

The Node.js backend sends structured requests to the Python AI microservice.

The AI service processes the request and returns structured results.

Figure 4.3 — System Communication Architecture



4.5 Database Design

Smart Event uses MySQL as the primary relational database.

Prisma ORM is used to provide type-safe interaction between the backend and database.

The database is designed around the major system domains:

- **Authentication and users.**
- **Events.**
- **Ticketing.**
- **Bookings.**
- **Payments.**
- **Speakers.**
- **B2B marketplace.**
- **RFQs.**

- **Quotations.**
- **Reviews.**
- **Notifications.**
- **AI interactions.**

4.5.1 Main Database Entities

The main entities include:

Authentication

- **User.**
- **Role.**
- **Customer.**
- **Organizer.**
- **Execution Company.**

Event Management

- **Event.**
- **Event Category.**
- **Event Schedule.**
- **Venue.**
- **Speaker.**
- **Sponsor.**

Ticketing

- **Ticket.**
- **Ticket Type.**
- **Booking.**
- **Payment.**
- **QR Ticket.**
- **Check-in.**
- **Waiting List.**

B2B Marketplace

- **Service.**
- **Service Category.**
- **Company Service.**
- **Service Request.**

- **Quotation.**
- **Contract.**
- **Review.**

AI and System Support

- **AI Conversation.**
- **AI Recommendation.**
- **User Preference.**
- **Notification.**
- **Event Analytics.**

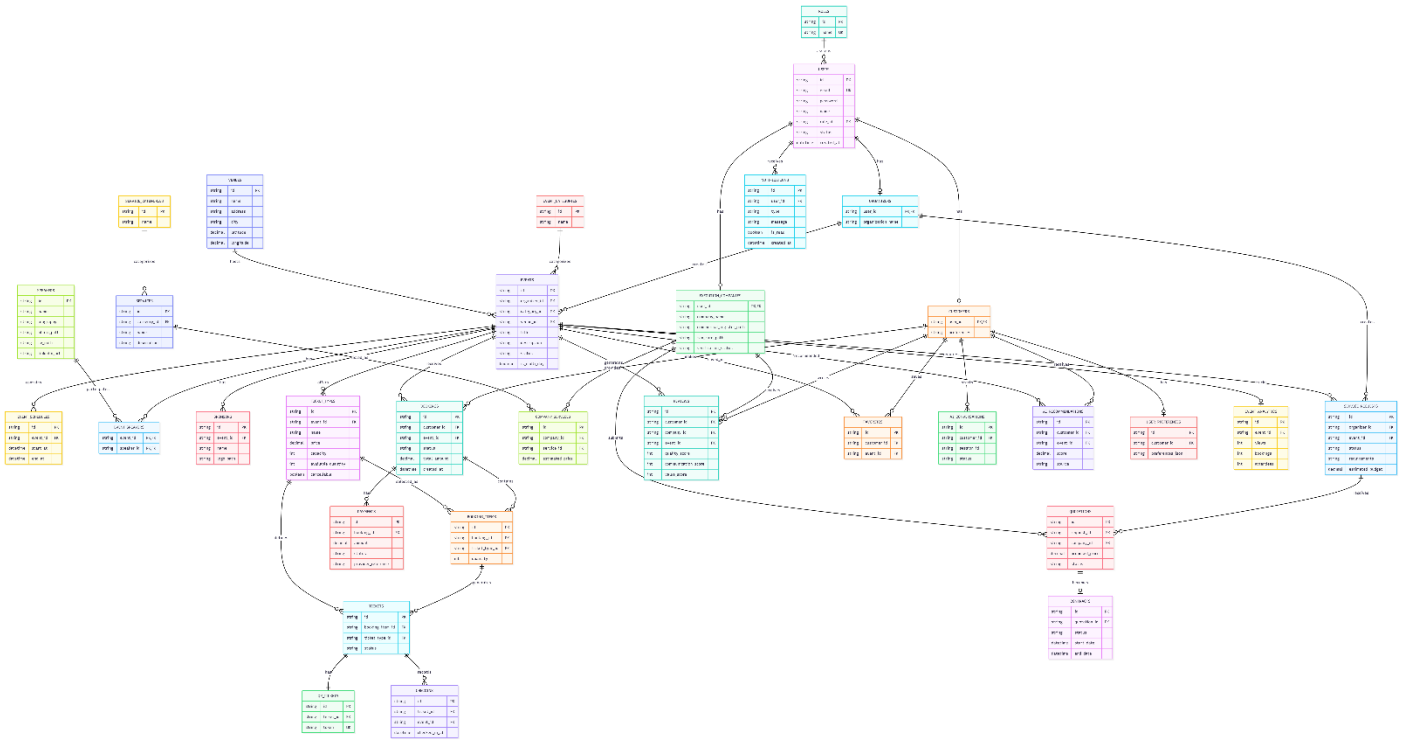
4.6 Entity Relationship Diagram

The ERD illustrates the relationships between the major entities in the Smart Event database.

The database must support relationships such as:

- **User → Role.**
- **Organizer → Events.**
- **Event → Schedules.**
- **Event → Speakers.**
- **Event → Ticket Types.**
- **Booking → Tickets.**
- **Ticket → QR Ticket.**
- **QR Ticket → Check-in.**
- **Execution Company → Services.**
- **Service Request → Quotations.**
- **Organizer → Service Requests.**
- **Execution Company → Quotations.**
- **Users → Reviews.**
- **Users → Notifications.**

Figure 4.4 — Entity Relationship Diagram



4.7 Data Dictionary

The database data dictionary defines the important attributes of the main entities.

Users

Attribute	Description
id	Unique user identifier
name	User name
email	User email
password	Securely stored password
roleId	User role
createdAt	Account creation date

Events

Attribute	Description
id	Unique event identifier
organizerId	Event owner
categoryId	Event category
title	Event title

Attribute	Description
description	Event description
location	Event location
status	Event status
createdAt	Creation date

Ticket Types

Attribute	Description
id	Ticket type identifier
eventId	Associated event
name	Ticket category
price	Ticket price
capacity	Maximum available quantity
cancellationPolicy	Cancellation configuration

Bookings

Attribute	Description
id	Booking identifier
userId	Attendee
eventId	Event
status	Booking status
totalAmount	Total booking value
createdAt	Booking date

Service Requests

Attribute	Description
id	Request identifier
organizerId	Request owner
eventId	Related event
requirements	Required services
budget	Expected budget

Attribute	Description
status	Request status

Quotations

Attribute	Description
id	Quotation identifier
requestId	Related RFQ
companyId	Execution company
price	Proposed price
details	Quotation details
status	Quotation status

4.8 UML Design

UML diagrams are used to represent the behavior and structure of the system.

4.8.1 Use Case Diagram

The Use Case Diagram represents the interactions between the four main actors and the system.

Actors

- **Attendee.**
- **Event Organizer.**
- **Execution Company.**
- **Super Admin.**

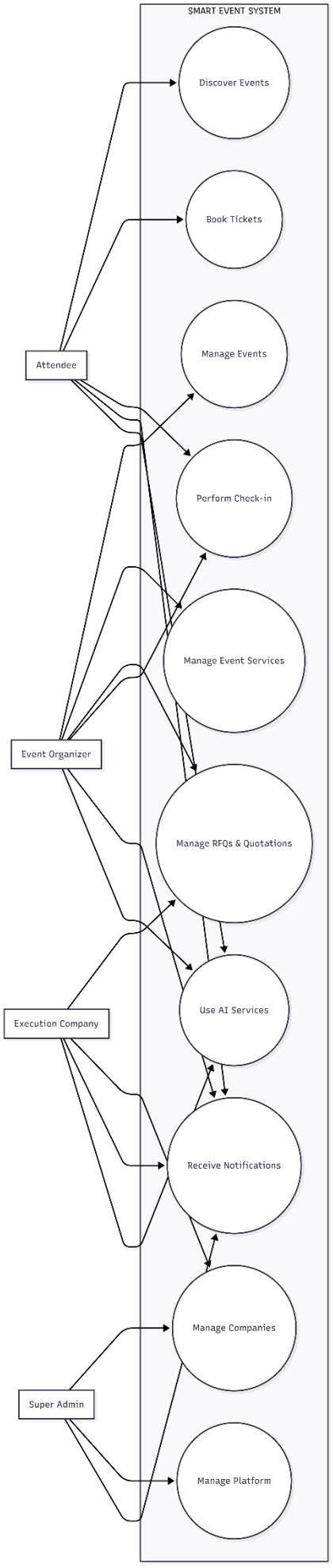


Figure 4.5 — Use Case Diagram

4.9 Use Case Specifications

UC-01 — Discover Event

Actor: Attendee

Description:
The attendee searches for available events using filters or the AI assistant.

Main Flow:

1. **Attendee opens event discovery.**
2. **Attendee enters search criteria.**
3. **System processes the request.**
4. **System retrieves suitable events.**
5. **System displays results.**

UC-02 — Book Event

Actor: Attendee

Description:
The attendee selects ticket types and quantities and creates a booking.

Main Flow:

1. **Attendee opens event.**
2. **Attendee selects ticket type.**
3. **Attendee selects quantity.**
4. **Attendee may select additional ticket types.**
5. **System calculates the total.**
6. **Booking is created.**
7. **Payment-processing workflow is initiated.**
8. **Booking status is updated.**
9. **Digital tickets are generated after successful confirmation.**

UC-03 — Create Event

Actor: Organizer

Main Flow:

1. **Organizer opens event management.**
2. **Organizer enters event information.**
3. **Organizer adds schedules.**
4. **Organizer adds speakers.**
5. **Organizer configures ticket types.**
6. **Organizer defines capacities.**
7. **Organizer submits event.**
8. **Event enters the required approval workflow.**

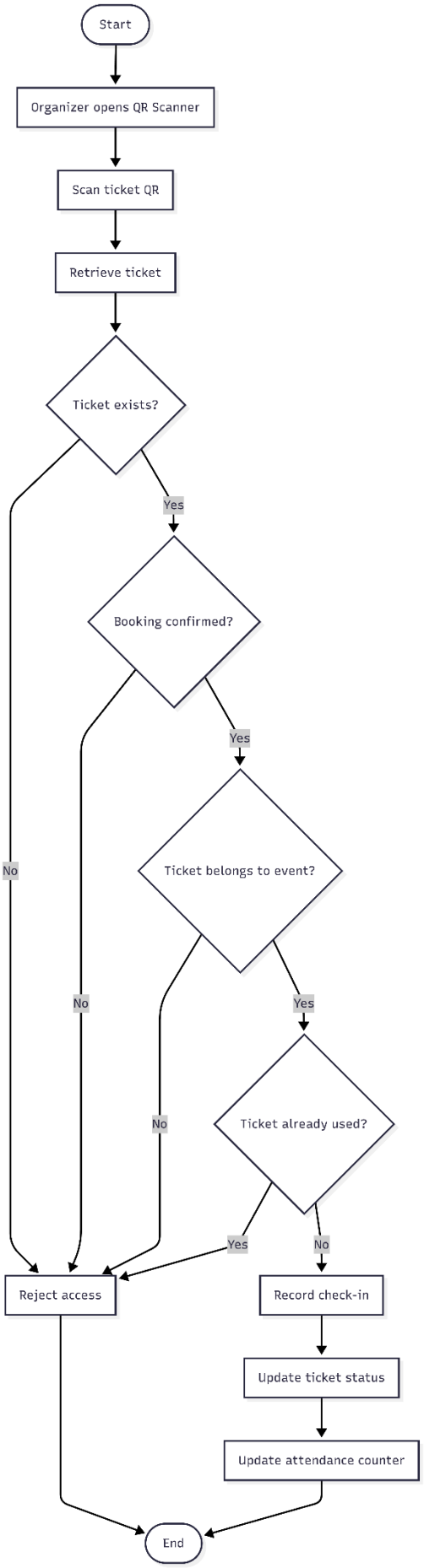
UC-04 — Scan Ticket

Actor: Organizer

Main Flow:

1. **Organizer opens check-in scanner.**
2. **Camera scans QR code.**
3. **System retrieves ticket information.**
4. **System validates the ticket.**
5. **System checks whether the ticket has already been used.**
6. **System records check-in if valid.**
7. **System displays the result.**

Figure 4.6 — Ticket Check-in Activity Diagram



UC-05 — Submit RFQ

Actor: Organizer

Main Flow:

1. **Organizer selects an event.**
2. **Organizer defines required services.**
3. **Organizer provides requirements and budget.**
4. **System creates an RFQ.**
5. **Matching vendors are identified.**
6. **RFQ becomes available to eligible execution companies.**

UC-06 — Submit Quotation

Actor: Execution Company

Main Flow:

1. **Company views available RFQs.**
2. **Company selects a suitable request.**
3. **Company prepares quotation.**
4. **Company submits quotation.**
5. **System records quotation.**
6. **Organizer receives a notification.**

UC-07 — Verify Execution Company

Actor: Super Admin

Main Flow:

1. **Company submits verification documents.**
2. **Admin reviews documents.**
3. **Admin approves or rejects the request.**
4. **System updates company verification status.**
5. **Company receives notification.**

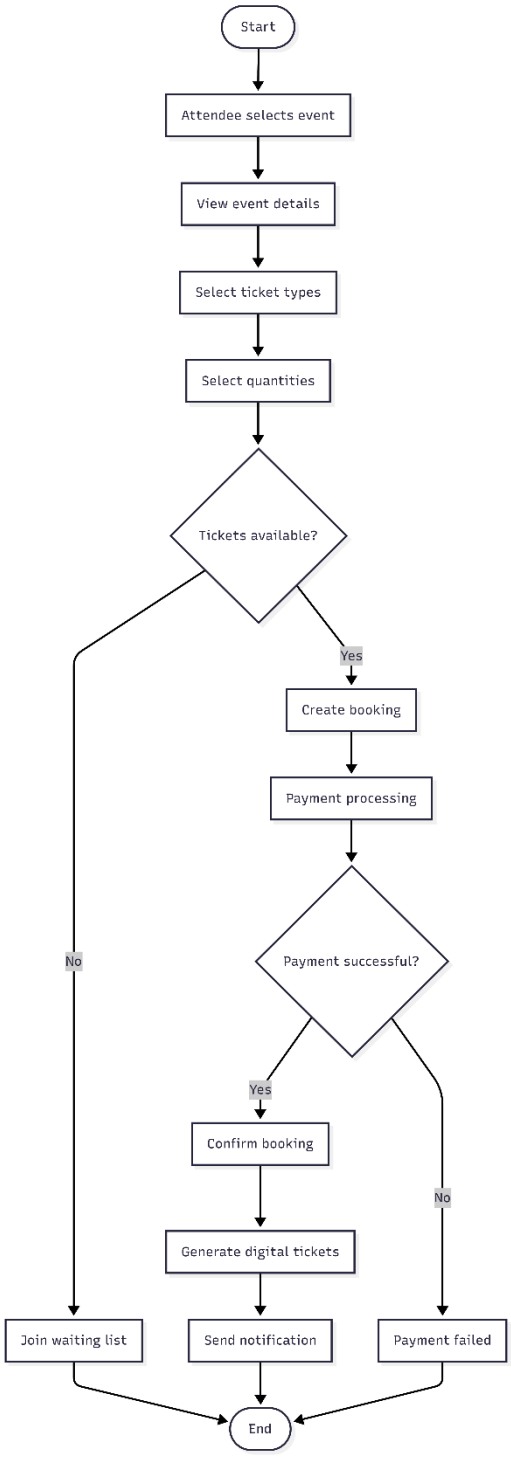
4.10 Activity Diagrams

Activity diagrams describe the sequence of activities for important system workflows.

4.10.1 Event Booking Activity

The booking process begins when the attendee selects an event and continues through ticket selection, booking creation, payment processing, and ticket generation.

Figure 4.7 — Event Booking Activity Diagram

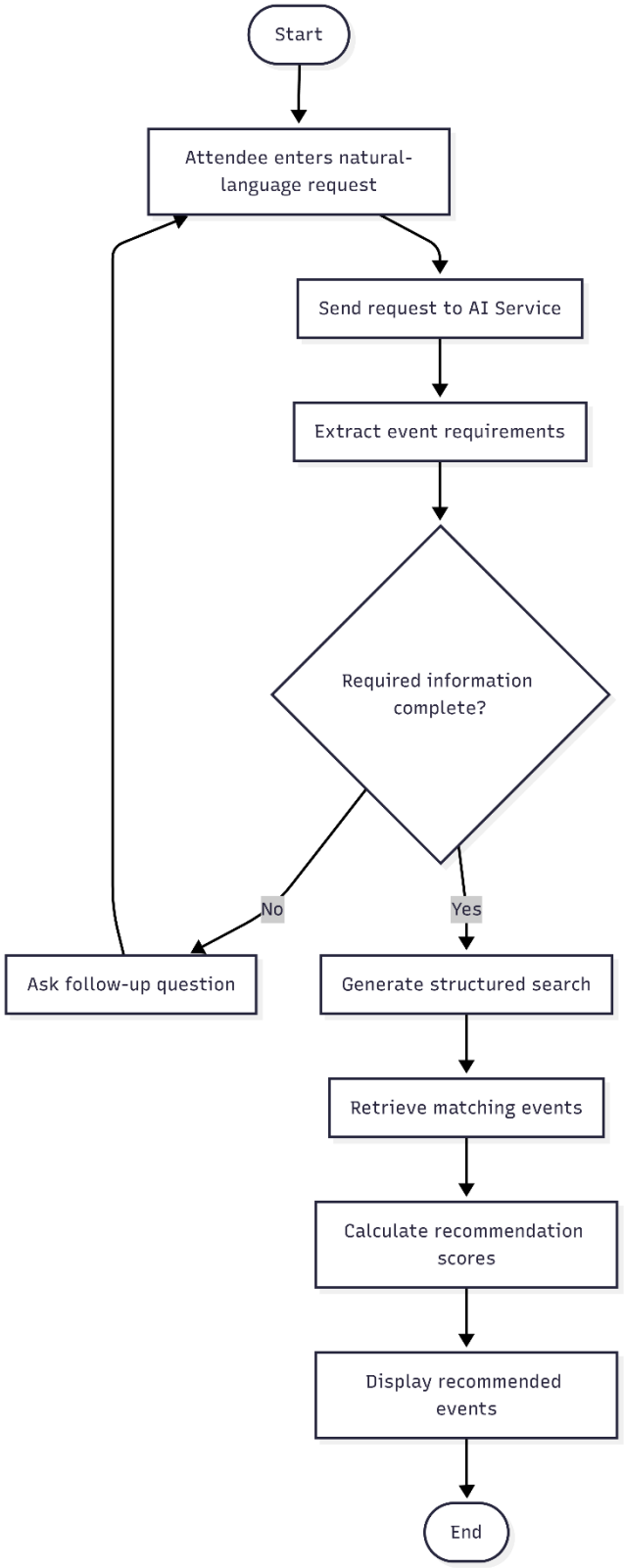


4.10.2 AI Event Discovery Activity

The AI discovery process follows:

User Input → Intent/Slot Extraction → Missing Information Detection → Follow-up → Structured Search → Recommendation

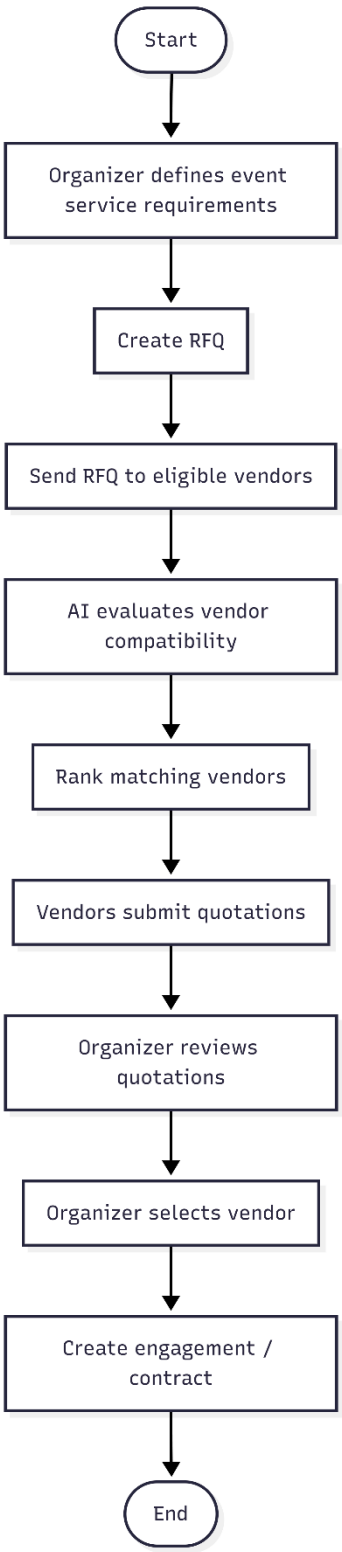
Figure 4.8 — AI Event Discovery Activity Diagram



4.10.3 Vendor Matching Activity

The organizer submits requirements, after which the AI service evaluates suitable companies based on the available matching criteria.

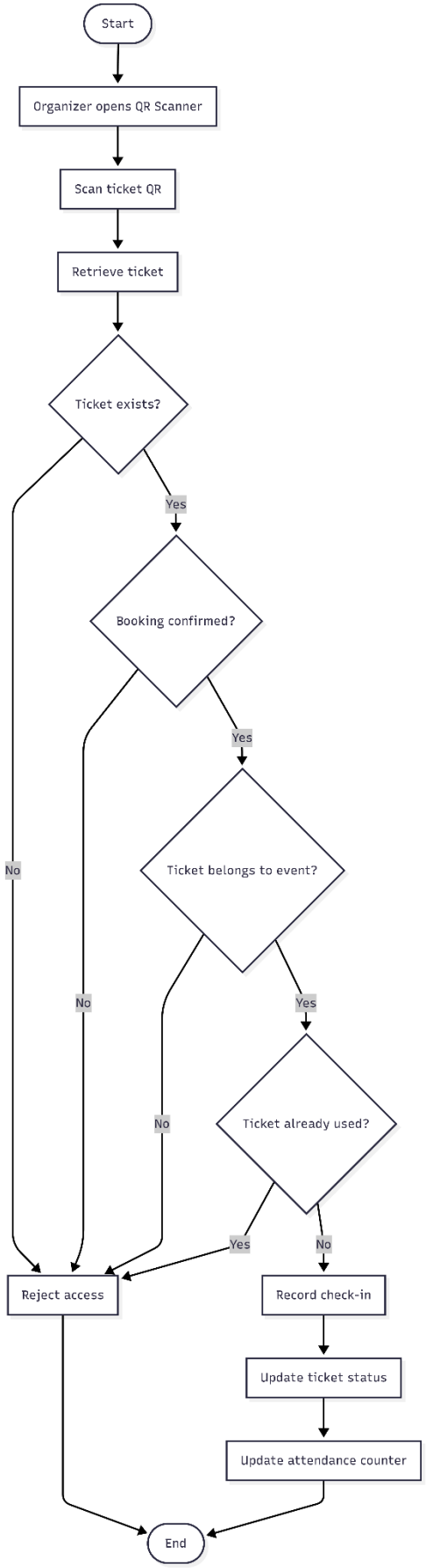
Figure 4.9 — Vendor Matching Activity Diagram



4.10.4 QR Check-in Activity

The check-in process verifies the ticket before granting event access.

Figure 4.10 — QR Check-in Activity Diagram



4.11 Sequence Diagrams

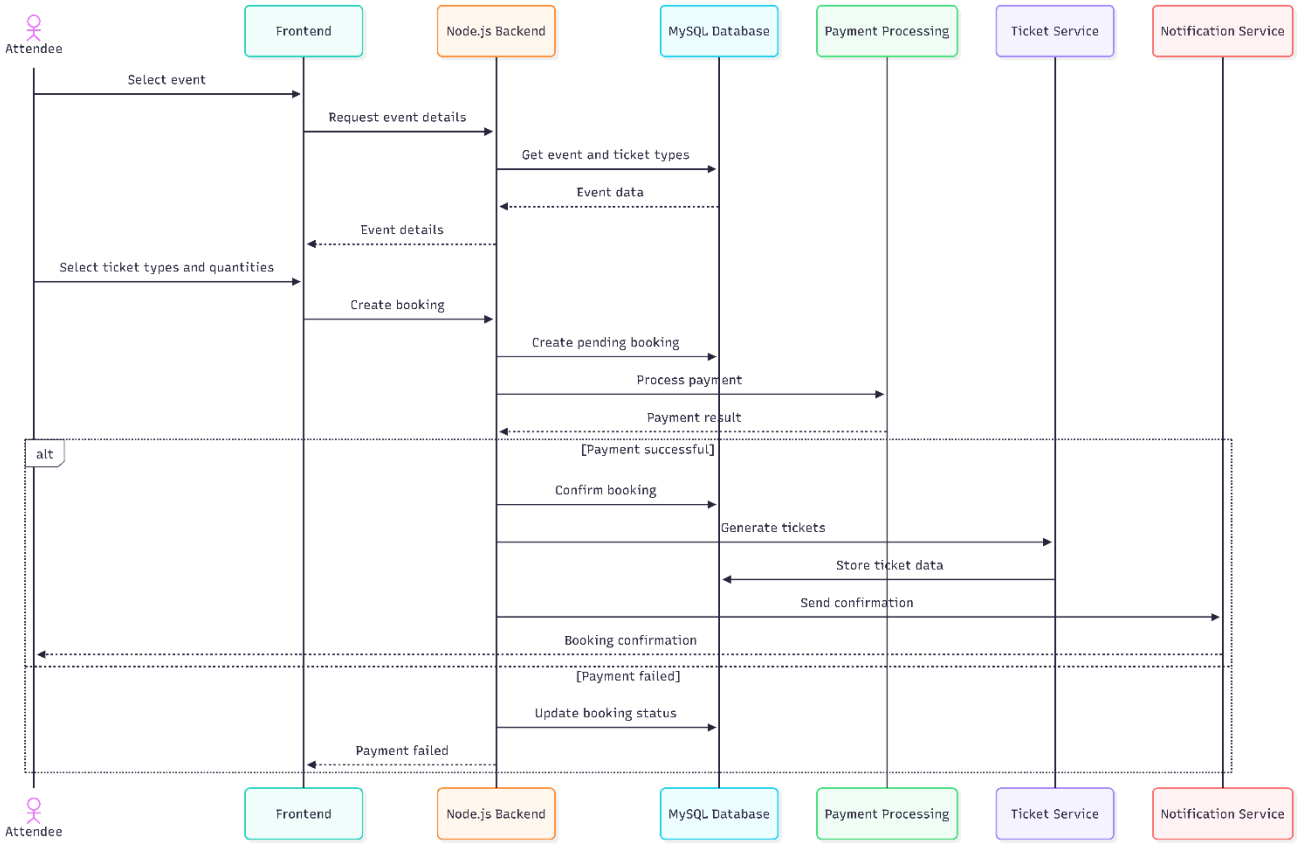
Sequence diagrams describe communication between system components during important operations.

4.11.1 Event Booking Sequence

The main participants are:

- **Attendee.**
- **Frontend.**
- **Backend.**
- **Database.**
- **Payment Processing Layer.**
- **Ticket Service.**

Figure 4.11 — Event Booking Sequence Diagram



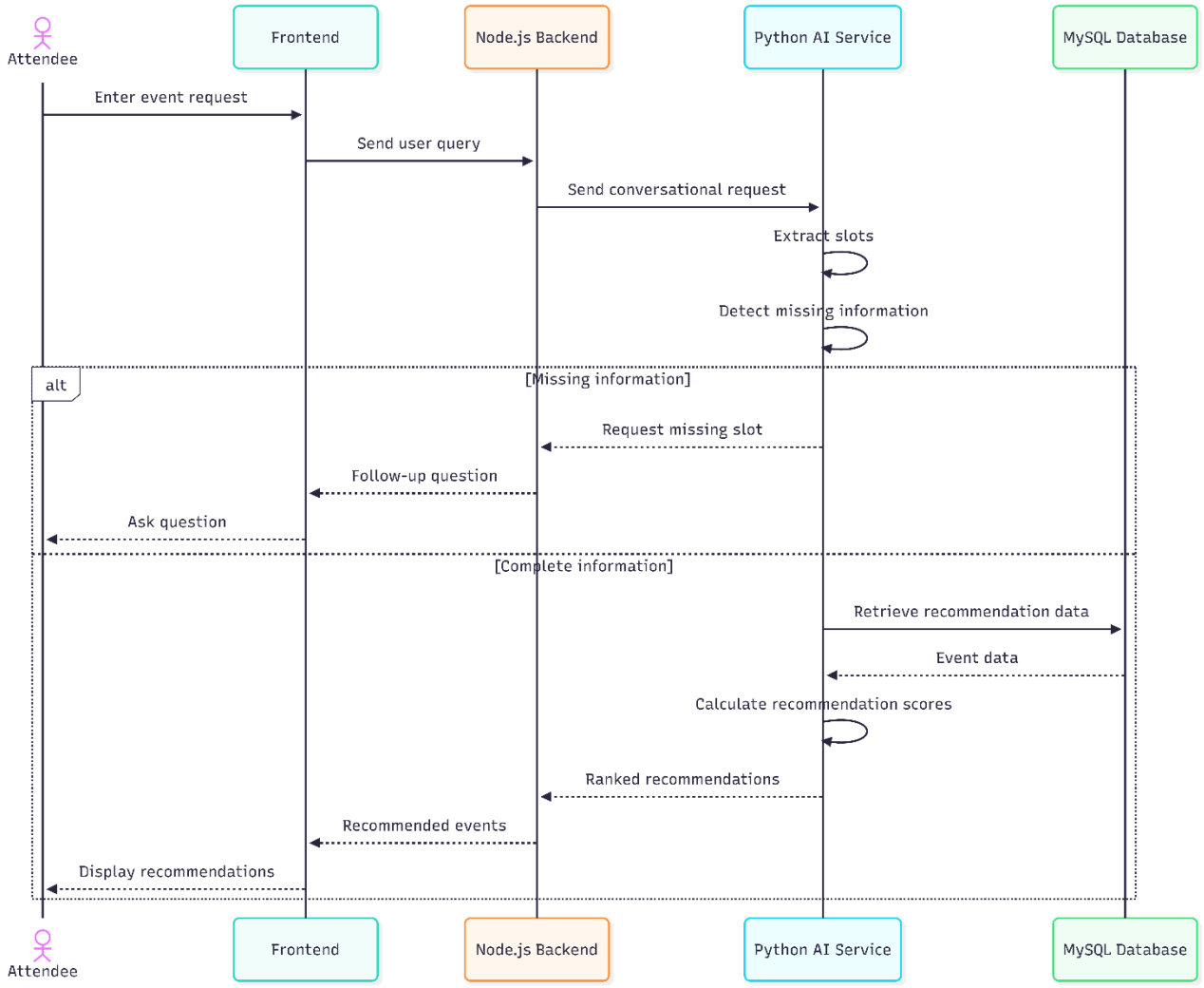
4.11.2 AI Recommendation Sequence

The main participants are:

- **Attendee.**

- **Frontend.**
- **Node.js Backend.**
- **Python AI Service.**
- **Database.**

Figure 4.12 — AI Recommendation Sequence Diagram

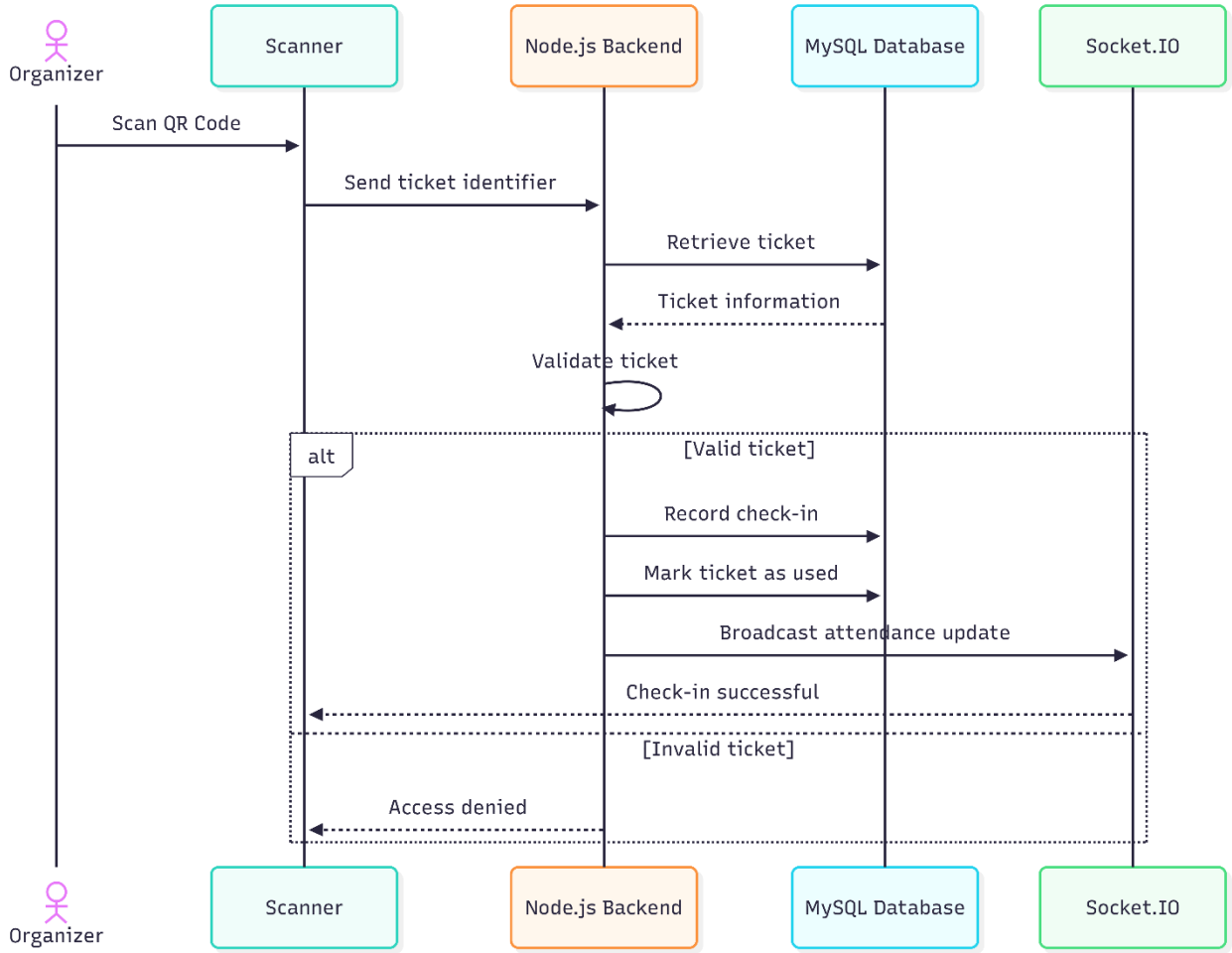


4.11.3 QR Check-in Sequence

The main participants are:

- **Organizer.**
- **Scanner Interface.**
- **Backend.**
- **Ticket Verification Service.**
- **Database.**
- **Socket.IO.**

Figure 4.13 — QR Check-in Sequence Diagram

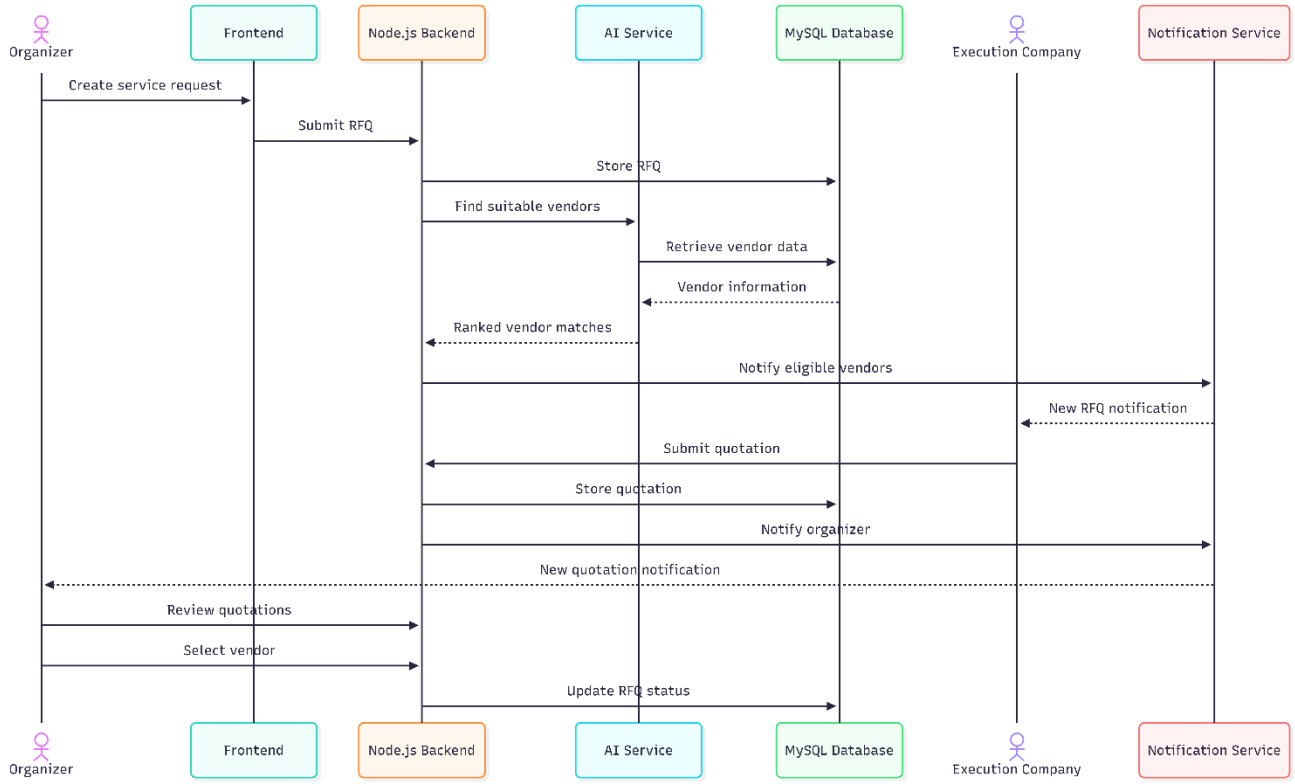


4.11.4 RFQ and Quotation Sequence

The main participants are:

- **Organizer.**
- **Frontend.**
- **Backend.**
- **AI Matching Service.**
- **Execution Company.**
- **Database.**

Figure 4.14 — RFQ and Quotation Sequence Diagram



4.12 Class Diagram

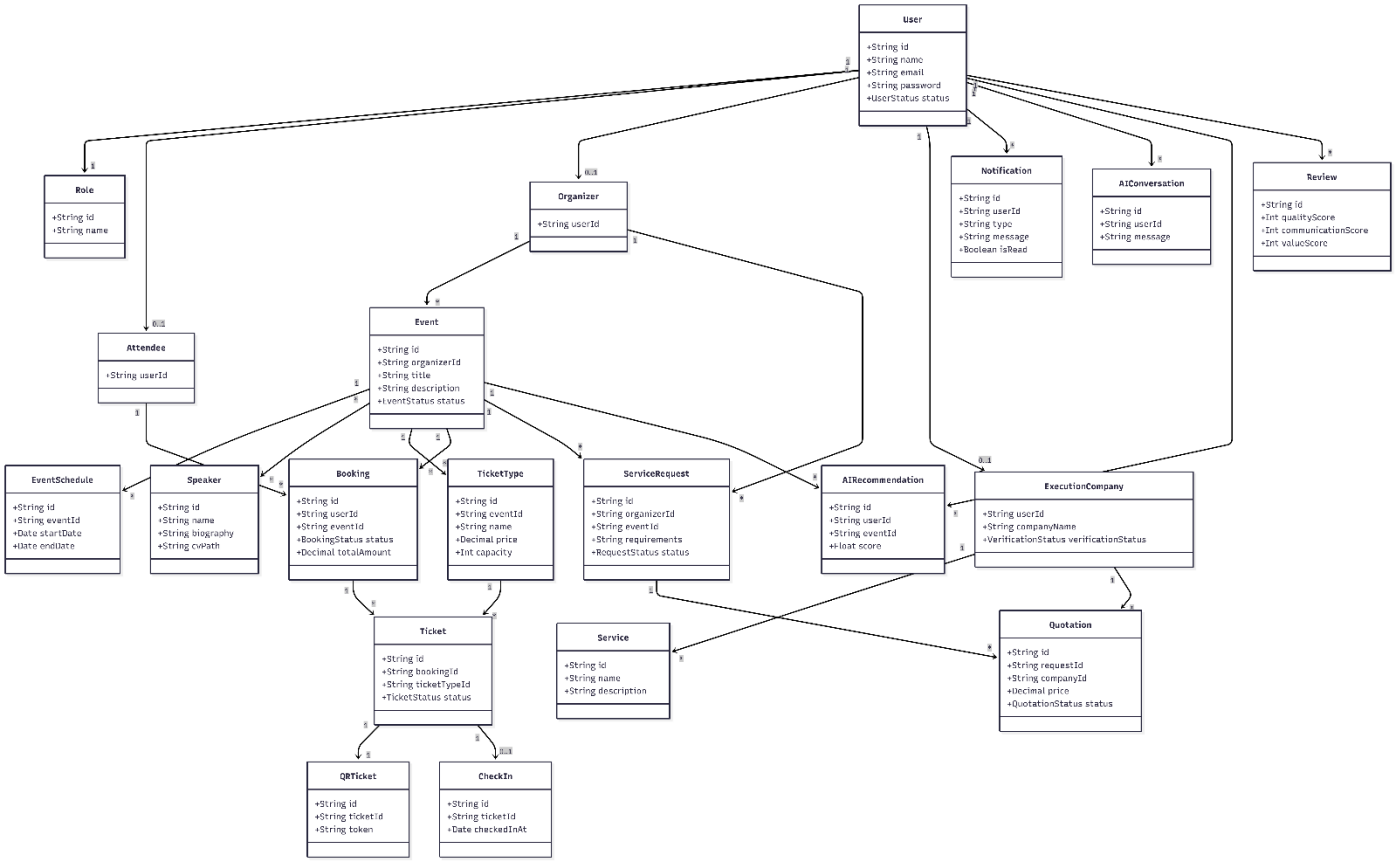
The Class Diagram represents the major classes and their relationships within the system.

Important classes include:

- **User.**
- **Role.**
- **Attendee.**
- **Organizer.**
- **ExecutionCompany.**
- **Event.**
- **EventSchedule.**
- **Speaker.**
- **TicketType.**
- **Booking.**
- **Ticket.**
- **QRTicket.**

- **CheckIn.**
- **Service.**
- **ServiceRequest.**
- **Quotation.**
- **Review.**
- **Notification.**
- **AIConversation.**
- **AIRecommendation.**

Figure 4.15 — Class Diagram



4.13 Payment Processing Design

Payment functionality is included in the system architecture as a processing workflow.

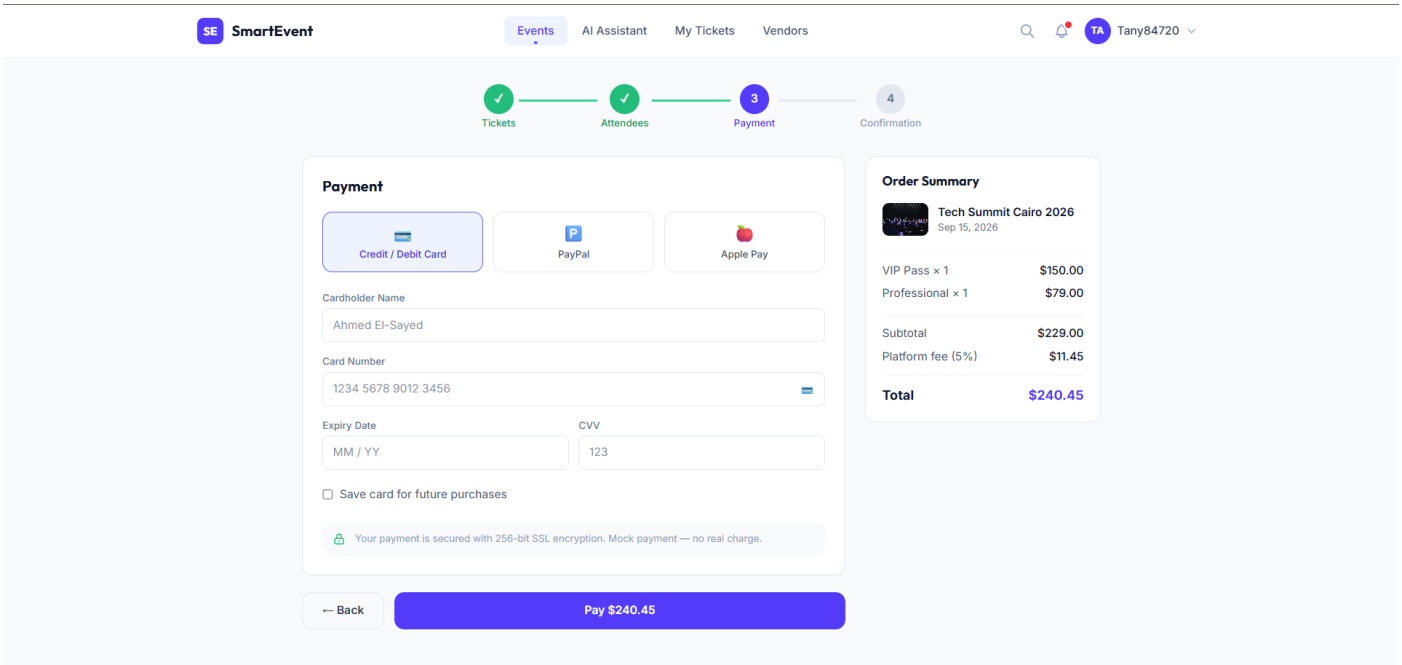
However, real payment gateway activation is not part of the initial implementation.

The architecture should therefore keep the payment component modular so that a real payment provider can be integrated later without changing the core booking architecture.

The conceptual process is:

Booking → Payment Processing → Payment Result → Booking Confirmation → Ticket Generation

Figure 4.16 — Payment Processing Workflow



4.14 Notification Design

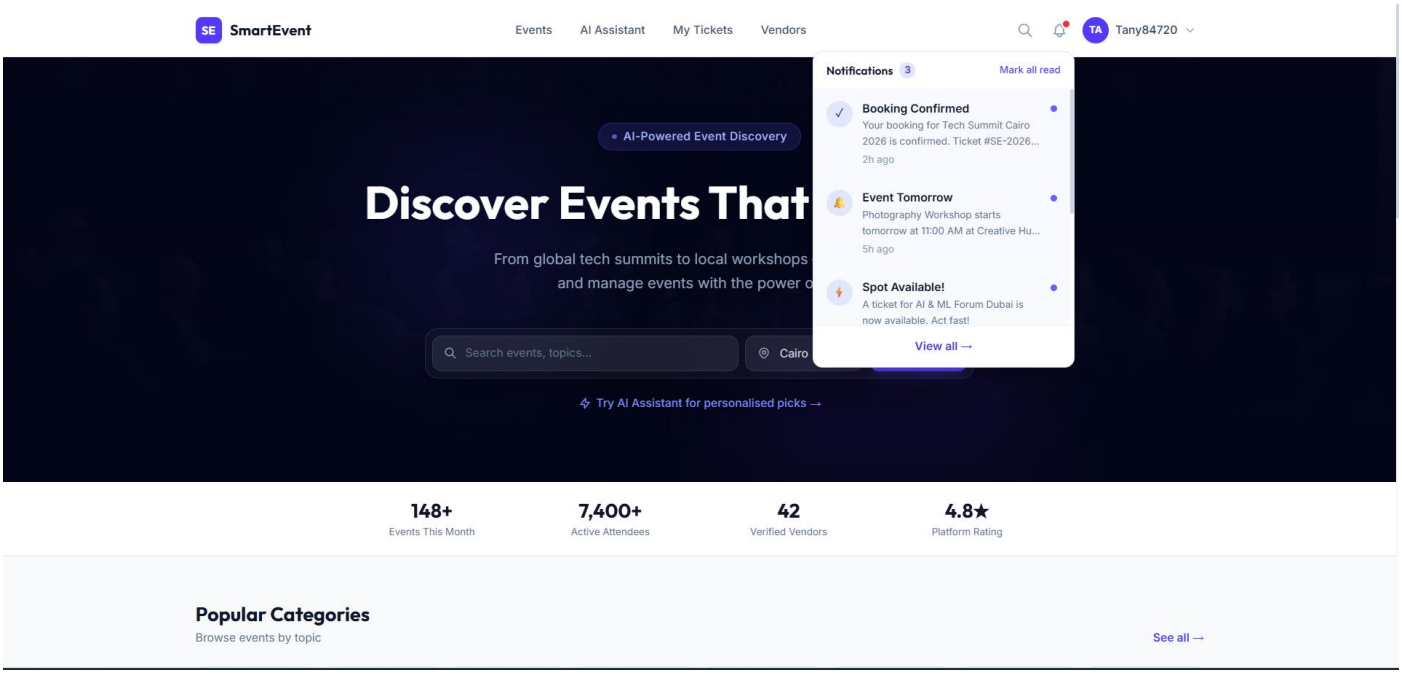
The system provides notifications for important events.

Examples include:

- **Booking confirmation.**
- **Booking cancellation.**
- **Ticket updates.**
- **Waiting-list updates.**
- **Event changes.**
- **RFQ updates.**
- **Quotation submission.**
- **Company verification.**
- **Administrative decisions.**

The notification functionality uses an external notification library/service rather than implementing the complete delivery infrastructure internally.

Figure 4.17 — Notification Flow



4.15 File and Media Storage Design

The system stores uploaded images and documents separately from the relational database.

The database stores the file path/reference, while the actual files are stored inside the configured upload storage.

Examples include:

- **Event images.**
- **User profile images.**
- **Speaker images.**
- **Speaker CVs.**
- **Company documents.**
- **Company portfolio files.**

The system should not store large binary files directly inside the main relational tables unless a specific requirement justifies doing so.

Figure 4.18 — File Storage Architecture

4.16 User Interface Design

The web interface is divided according to the four system actors.

4.16.1 Attendee Interface

Main pages include:

- **Home.**
- **Event Discovery.**
- **AI Assistant.**
- **Event Details.**
- **Booking.**
- **My Bookings.**
- **My Tickets.**
- **Ticket Details.**
- **Notifications.**
- **Profile.**

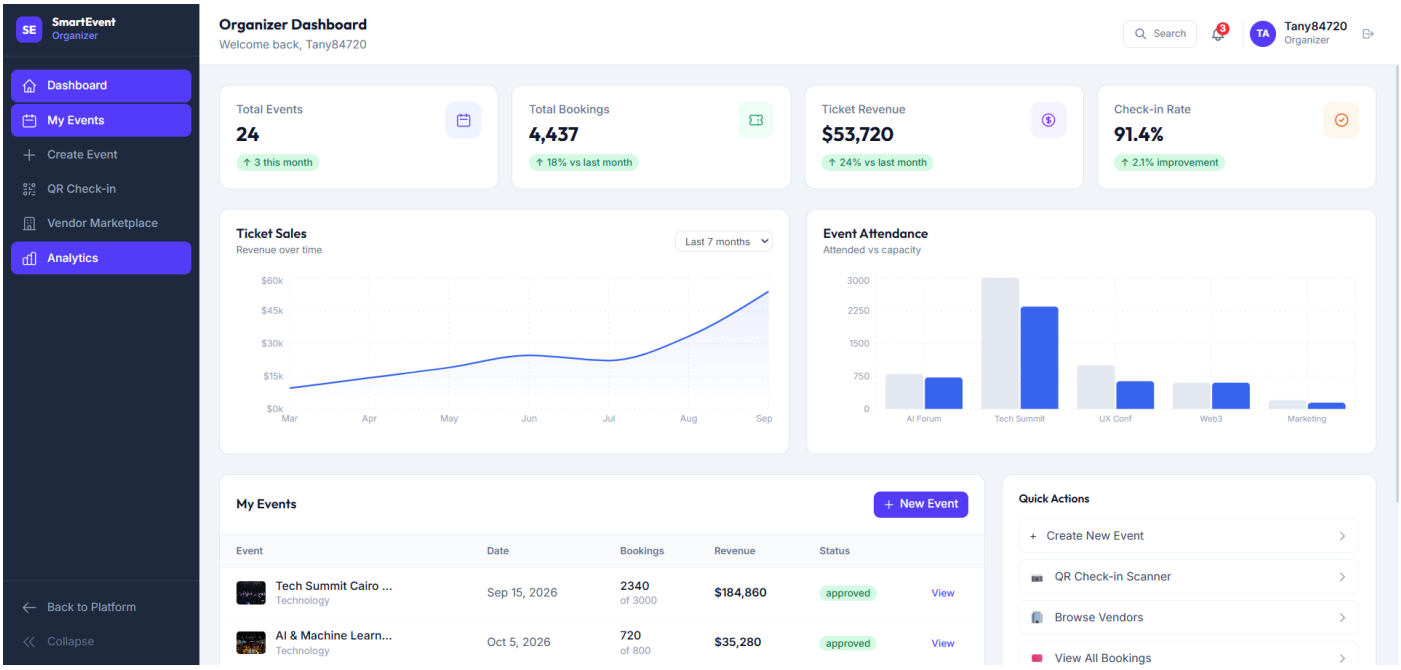
Figure 4.19 — Attendee Interface / Main Navigation

4.16.2 Organizer Interface

Main pages include:

- **Organizer Dashboard.**
- **My Events.**
- **Create Event.**
- **Edit Event.**
- **Event Details.**
- **Ticket Management.**
- **Attendee Management.**
- **QR Check-in.**
- **Speakers.**
- **Sponsors.**
- **Service Requests.**
- **RFQs.**
- **Quotations.**
- **Notifications.**
- **Organizer Profile.**

Figure 4.20 — Organizer Dashboard



4.16.3 Execution Company Interface

Main pages include:

- **Company Dashboard.**
- **Company Profile.**
- **Verification.**
- **Services.**
- **Available RFQs.**
- **RFQ Details.**
- **Submit Quotation.**
- **Quotations.**
- **Active Contracts.**
- **Reviews.**
- **Notifications.**

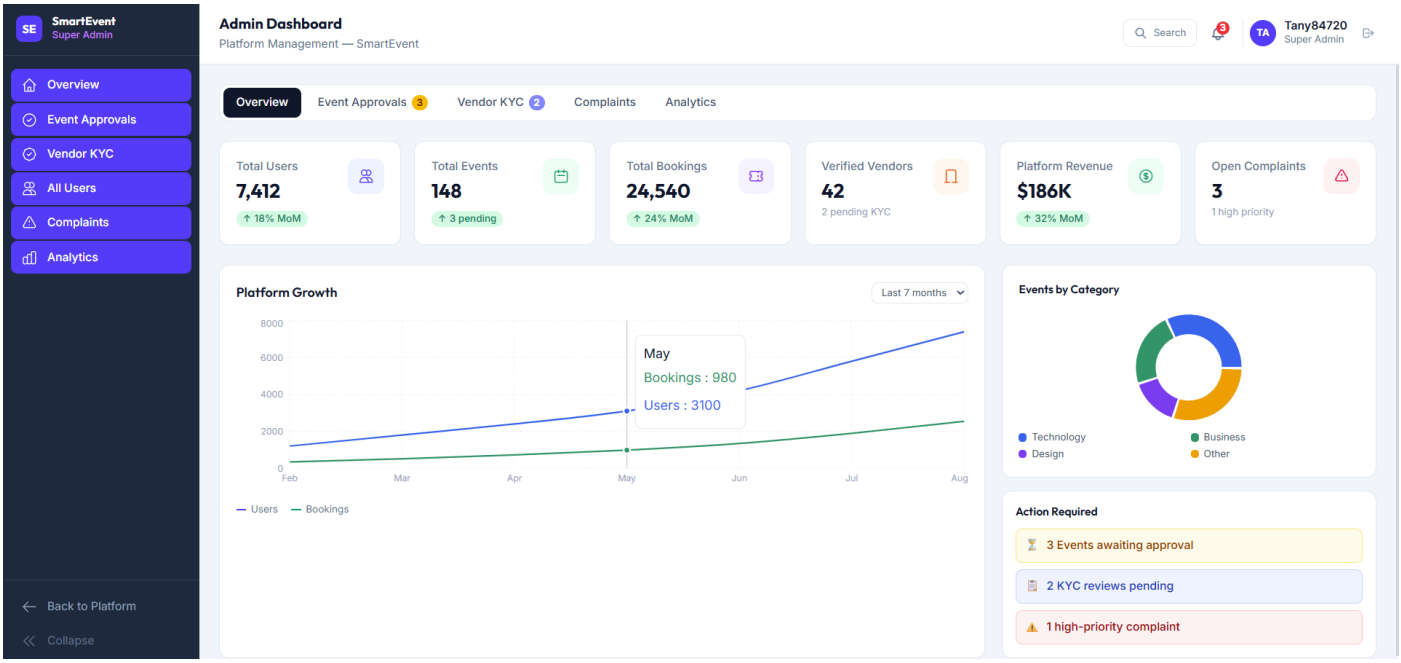
Figure 4.21 — Execution Company Dashboard

4.16.4 Super Admin Interface

Main pages include:

- **Admin Dashboard.**
- **Users.**
- **Organizers.**
- **Execution Companies.**
- **Company Verification.**
- **Events.**
- **Event Approval.**
- **Bookings.**
- **Payments.**
- **Disputes.**
- **Reviews.**
- **Notifications.**
- **Analytics.**

Figure 4.22 — Super Admin Dashboard



4.17 Mobile Application Design

The mobile application is considered a future expansion after completing the web application.

The backend architecture is designed so that the future mobile application can consume the same REST APIs and backend services without requiring a separate business logic implementation.

Figure 4.23 — Future Mobile Application Architecture

4.18 Chapter Summary

This chapter presented the design of the proposed Smart Event system.

The system architecture separates the frontend, core backend, database, AI microservice, and supporting services. The database design defines the main entities required for event management, ticketing, B2B services, AI functionality, and system administration.

The UML and workflow designs describe the interaction between the system actors and the major system components.

The design also provides a foundation for the implementation presented in Chapter 5, while maintaining extensibility for future capabilities such as mobile applications, real payment gateways, and external event discovery APIs.